TUGAS BESAR 1
IF3070 DASAR INTELIGENSI ARTIFISIAL

Ship Loading Optimizer

Dosen Pengampu:
Ir. Windy Gambetta, M.B.A.



Disusun Oleh:

Keira Amelie Hanayesha	18224003
Ellaine Juvina	18224071
Firanti Naulia Fasya	18224119

Program Studi Sistem dan Teknologi Informasi
Sekolah Teknik Elektro dan Informatika – Komputasi
Institut Teknologi Bandung
2026

DAFTAR ISI

DAFTAR ISI **1**
DAFTAR TABEL **2**
DAFTAR GAMBAR **3**
BAB 1 DESKRIPSI PERSOALAN **4**
BAB 2 PEMBAHASAN **5**
1. Metodologi 5
1.1. Objectives 5
1.2. Constraint 6
1.3. Dataset 7
1.4. Experiments 7
2. Implementasi Algoritma Local Search 8
2.1. Steepest-Ascent Hill Climbing 11
2.2. Sideways Move Hill Climbing 17
2.3. Stochastic Hill Climbing 23
2.4. Random Restart Hill Climbing 28
2.5. Simulated Annealing 34
2.6. Genetic Algorithm 42
2.7. ETA-Weighted Objective 61
2.8. Analisis Eksperimen 64
2.8.1. Kualitas Solusi 67
2.8.2. Final State versus Best State 67
2.8.3. Kedekatan terhadap Global Optimum 68
2.8.4. Konsistensi 69
2.8.5. Pengaruh Jumlah Generasi Genetic Algorithm 70
2.8.6. Pengaruh Ukuran Populasi Genetic Algorithm 70
2.8.7. Mengapa Simulated Annealing Mendapatkan Solusi Terbaik 70
2.8.8. Ancaman terhadap Validitas 71
BAB 3 KESIMPULAN DAN SARAN **72**
PEMBAGIAN TUGAS **74**
REFERENSI **75**
LAMPIRAN FORM PENGGUNAAN GENERATIVE AI **76**

DAFTAR TABEL

Tabel 1.3.a: Karakteristik Dataset synthetic_10×20 7
Tabel 2.a: Tabel Penjelasan Fungsi dan Kelas 8
Tabel 2.1.a: Tabel Kondisi Terminasi 13
Tabel 2.1.b: Hasil Steepest Hill Climbing 14
Tabel 2.1.c: Visualisasi Hasil Steepest Hill Climbing 15
Tabel 2.2.a: Tabel Parameter Sideways Move Hill Climbing 18
Tabel 2.2.b: Tabel Kondisi Terminasi 19
Tabel 2.2.c: Hasil Sideways Move Hill Climbing 20
Tabel 2.2.d: Visualisasi Hasil Sideways Move Hill Climbing 20
Tabel 2.3.a: Tabel Parameter Stochastic Hill climbing 23
Tabel 2.3.b: Tabel Kondisi Terminasi 25
Tabel 2.3.c: Hasil Stochastic Hill Climbing 25
Tabel 2.3.d: Visualisasi Hasil Stochastic Hill Climbing 26
Tabel 2.4.a: Tabel Parameter Random Restart Hill Climbing 29
Tabel 2.4.b: Tabel Kondisi Terminasi 30
Tabel 2.4.c: Hasil Random Restart Hill Climbing 31
Tabel 2.4.d: Visualisasi Hasil Random Restart Hill Climbing 32
Tabel 2.5.a: Tabel Parameter Simulated Annealing 35
Tabel 2.5.b: Tabel Kondisi Terminasi 36
Tabel 2.5.c: Tabel Variabel Stuck 37
Tabel 2.5.d: Hasil Simulated Annealing (Spesifikasi Wajib) 39
Tabel 2.5.e: Visualisasi Hasil Simulated Annealing 39
Tabel 2.6.a: Tabel Deskripsi Fungsi pada Genetic Algorithm 42
Tabel 2.6.b: Studi Parameter Genetic Algorithm 53
Tabel 2.6.c: Trial Genetic Algorithm pada Konfigurasi Terpilih 56
Tabel 2.6.d: Visualisasi Initial State dan Best-Ever State Genetic Algorithm pada Populasi 40 dan 100 Generasi 56
Tabel 2.6.e: Perbandingan Final State dan Best-ever State Genetic Algorithm pada Populasi 40 dan 100 Generasi 60
Tabel 2.7.a: Perbandingan hasil optimasi dengan ETA-Weighted Objective 62
Tabel 2.7.b: Visualisasi hasil optimasi dengan ETA-Weighted Objective 63
Tabel 2.8.a: Perbandingan Hasil Ketiga Algoritma 66
Tabel 2.8.1.a: Perbandingan Kualitas Solusi 68

DAFTAR GAMBAR

Gambar 2.6.a. Pengaruh jumlah generasi terhadap mean best-ever ShippingFee..... 54
Gambar 2.6.b. Pengaruh ukuran populasi terhadap mean best-ever ShippingFee.....55
Gambar 2.6.c. Konvergensi Genetic Algorithm pada population size 40 dan 100 generations..... 59
Gambar 2.8.a. Mean best-ever ShippingFee Hill Climbing, Simulated Annealing, dan Genetic Algorithm beserta sample standard deviation..... 66
Gambar 2.8.b. Runtime tercatat untuk Hill Climbing, Simulated Annealing, dan Genetic Algorithm..... 67

BAB 1
DESKRIPSI PERSOALAN

PT Owen Intim Line dalam studi kasus mengoperasikan kapal kargo pengangkut kendaraan dan menerima sejumlah kendaraan dengan karakteristik dimensi, berat, ShippingFee, dan ETA yang berbeda-beda. Proses penataan manual berpotensi tidak memanfaatkan ruang geladak secara optimal serta dapat menghasilkan pemilihan kendaraan yang kurang menguntungkan. Oleh karena itu, dibutuhkan sistem pendukung keputusan yang dapat mengevaluasi sejumlah konfigurasi pemuatan dan menentukan kombinasi kendaraan beserta tata letaknya secara otomatis. Permasalahan resmi memang meminta sistem memilih kendaraan, koordinat, dan orientasi pada grid dua dimensi dengan tujuan memaksimalkan ShippingFee.

Persoalan ini memiliki ruang solusi yang besar karena keputusan tidak hanya berupa apakah suatu kendaraan dimuat, tetapi juga koordinat serta orientasinya. Selain itu, setiap konfigurasi harus diperiksa terhadap beberapa constraint secara bersamaan. Oleh sebab itu, tugas menggunakan pendekatan *local search* dan metaheuristik, yaitu satu varian *Hill Climbing*, *Simulated Annealing*, serta *Genetic Algorithm*.

BAB 2
PEMBAHASAN

1. Metodologi

1.1. *Objectives*

Fungsi objektif merupakan komponen utama dalam pendekatan local search karena digunakan untuk mengevaluasi kualitas suatu solusi dan menentukan arah pencarian menuju konfigurasi yang lebih baik. Dalam permasalahan optimasi penempatan kendaraan pada kapal, fungsi objektif memberikan nilai terhadap suatu state berdasarkan kendaraan yang berhasil dimuat dengan tetap memperhatikan seluruh batasan penempatan.

Pada implementasi ini, digunakan dua fungsi objektif, yaitu ShippingFee Maximization sebagai fungsi objektif utama dan ETA-Weighted ShippingFee sebagai fungsi objektif alternatif. Keduanya menggunakan pendekatan maksimasi, sehingga semakin besar nilai fungsi objektif yang diperoleh, semakin baik kualitas solusi berdasarkan kriteria evaluasi yang dipilih.

a. ShippingFee Maximization

Fungsi objektif utama bertujuan memaksimalkan total ShippingFee dari seluruh kendaraan yang berhasil ditempatkan pada geladak kapal. Setiap kendaraan memiliki nilai ShippingFee tertentu yang diperoleh apabila kendaraan tersebut dimuat. Kendaraan yang tidak dimuat tidak memberikan kontribusi terhadap nilai fungsi objektif.

$$f_{fee}(S) = \sum_{i=1}^{n} x_i F_i$$

Dengan demikian, nilai fungsi objektif hanya dipengaruhi oleh kendaraan yang dimuat dan tidak secara langsung dipengaruhi oleh posisi maupun orientasinya. Posisi dan orientasi tetap berperan penting karena menentukan apakah suatu kombinasi kendaraan dapat ditempatkan secara valid.

b. ETA-Weighted ShippingFee

Sebagai pengembangan dari fungsi objektif utama, diimplementasikan fungsi objektif alternatif yang mempertimbangkan Estimated Time of Arrival (ETA). Dalam pendekatan ini, ETA diasumsikan sebagai jumlah hari tersisa hingga kedatangan kendaraan. Oleh karena itu, kendaraan dengan nilai ETA lebih kecil dianggap memiliki urgensi lebih tinggi dan diberikan bobot tambahan dalam perhitungan nilai objektif.

Fungsi objektif alternatif tetap mempertahankan nilai ShippingFee sebagai komponen dasar, tetapi memberikan faktor pengali berdasarkan tingkat urgensi masing-masing kendaraan.

$$f_{ETA}(S) = \sum_{i=1}^{n} x_i F_i \left(1 + \omega \frac{ETA_{max} - ETA_i}{ETA_{max}}\right)$$

Nilai urgensi akan semakin besar ketika ETA kendaraan semakin kecil. Dengan demikian, kendaraan yang memiliki ETA lebih kecil memperoleh faktor pengali yang lebih tinggi dibandingkan kendaraan dengan ETA besar.

1.2. *Constraint*

Selain fungsi objektif, optimasi penempatan kendaraan harus memperhatikan sejumlah batasan yang menentukan validitas suatu solusi. Batasan tersebut diterapkan pada seluruh algoritma agar konfigurasi yang dihasilkan dapat direalisasikan pada geladak kapal.

a. No Overlap

Setiap kendaraan yang dimuat harus menempati area grid yang tidak bertumpuk dengan kendaraan lainnya. Satu sel pada geladak kapal hanya boleh ditempati oleh satu kendaraan.

$$\forall i \neq j,\ Cells(v_i) \cap Cells(v_j) = \emptyset$$

Batasan ini memastikan tidak terdapat dua kendaraan yang menggunakan area geladak yang sama, sehingga seluruh penempatan kendaraan bersifat eksklusif.

b. Ship Boundary

Seluruh bagian kendaraan yang dimuat harus berada di dalam dimensi geladak kapal.

$$Cells(v_i) \subseteq Deck$$

Orientasi kendaraan menentukan pertukaran dimensi horizontal dan vertikal yang digunakan dalam perhitungan area penempatan. Oleh karena itu, setiap perubahan orientasi harus diikuti pemeriksaan kembali terhadap batas geladak.

c. Maximum Weight Capacity

Total berat seluruh kendaraan yang dimuat tidak boleh melebihi kapasitas maksimum kapal.

$$\sum_{v_i \in L(s)} weight_i \leq MaxCapacity.$$

Batasan ini memastikan bahwa kendaraan yang berhasil ditempatkan secara spasial juga memenuhi kemampuan angkut kapal.

1.3. *Dataset*

Tabel 1.3.a: Karakteristik *Dataset* synthetic_10×20

Atribut	Nilai
Lebar kapal	10
Panjang kapal	20
Luas geladak	200 sel
Max capacity	500
Jumlah kendaraan	30
Total luas kendaraan	206 sel
Total berat kendaraan	304
Total ShippingFee seluruh kendaraan	4780
Rentang ETA	1–10

Total berat seluruh kendaraan berada di bawah kapasitas. Sebaliknya, total luas lebih besar daripada sel kapal. Hal ini menunjukkan bahwa pada dataset tersebut, keterbatasan tata ruang merupakan sumber *constraint* yang lebih langsung dibandingkan kapasitas berat, walaupun bentuk geometris dan fragmentasi ruang membuat luas total saja tidak cukup untuk menentukan *feasibility*.

1.4. *Experiments*

Eksperimen menggunakan *seed* 42, 43, dan 44 dengan LOAD_PROBABILITY ditetapkan 0,5. *Hill Climbing* memiliki HC_MAX_ITERATIONS = 1000, sedangkan *Simulated Annealing* menggunakan temperatur awal 1000,0, cooling rate 0,9995, temperatur minimum 0,1, dan maksimum 50.000 iterasi. Berdasarkan *output* aktual, *Simulated Annealing* berhenti setelah 18.417 iterasi akibat tercapainya temperatur minimum. *Genetic Algorithm* menggunakan *mutation rate* 0,1 dengan dua studi parameter:

- Populasi tetap = 40, generasi = {50, 100, 150}, dan
- Generasi tetap = 100, populasi = {20, 40, 50}.

Setiap konfigurasi menggunakan ketiga *seed* yang sama, menghasilkan 18 *run Genetic Algorithm*. *Manifest* eksperimen mencatat *dataset fingerprint* SHA-256, parameter, *seed*, dan jumlah *run* sehingga hasil dapat diverifikasi terhadap *dataset* yang sama.

Analisis menggunakan *best solution encountered* sebagai metrik utama kualitas pencarian karena metrik tersebut menjawab pertanyaan “solusi terbaik apa yang

berhasil ditemukan algoritma". Untuk transparansi, final_state tetap dilaporkan secara terpisah, terutama karena desain *Genetic Algorithm* tanpa *elitism* dapat menyebabkan *final population* lebih buruk daripada solusi terbaik yang pernah muncul. Metode statistik yang digunakan bersifat deskriptif, yaitu rata-rata, s*ample standard deviation*, $[s\sqrt{\frac{\sum_{i=1}^{n}(x_i-\bar{x})^2}{n-1}},]$, *coefficient of variation* $(CV = 100s/\bar{x})$, dan selisih berpasangan berdasarkan *seed*. Karena hanya tersedia tiga *trial* independen per konfigurasi, laporan ini tidak menggunakan uji signifikansi parametrik sebagai dasar klaim keunggulan algoritma.

2. Implementasi Algoritma Local Search

Implementasi algoritma Local Search menggunakan beberapa model utama, yaitu `Vehicle`, `Ship`, `Placement`, `Orientation`, dan `State`. Model-model tersebut merepresentasikan kendaraan, kapal, posisi kendaraan, orientasi penempatan, serta konfigurasi penempatan seluruh kendaraan. Solusi direpresentasikan oleh `State`, yang memetakan setiap `vehicle_id` ke objek `Placement` jika kendaraan dimuat ke kapal, atau None jika kendaraan tidak dimuat.

Tabel 2.a: Tabel Penjelasan Fungsi dan Kelas

Komponen	Fungsi/Kelas	Penjelasan
Representasi Kendaraan	`Vehicle`	Menyimpan atribut `id, width, length, shipping_fee, weight`, dan `eta` sebagai informasi kendaraan.
Representasi Kapal	`Ship`	Menyimpan dimensi kapal melalui `width` dan `length`, serta kapasitas berat maksimum melalui `max_capacity`.
Representasi Penempatan	`Placement, Orientation`	`Placement` menyimpan koordinat x, y, dan orientasi kendaraan. `Orientation` menentukan orientasi `HORIZONTAL` atau `VERTICAL`, yang memengaruhi dimensi penempatan.
Representasi *State*	`State`	Menyimpan `placements`, yaitu pemetaan setiap `ID` kendaraan ke `Placement` atau `None`. Kelas ini juga menyediakan fungsi untuk menyalin state dan memperoleh daftar kendaraan yang dimuat atau tidak dimuat

Komponen	Fungsi/Kelas	Penjelasan
Initializer	`generate_random_state()`	Membentuk solusi awal secara acak dengan mengacak urutan kendaraan dan mencoba menempatkannya pada posisi yang memenuhi batasan. Parameter `load_prob` menentukan peluang kendaraan dicoba untuk dimuat.
Constraint	`is_valid_state()`	Memvalidasi kelengkapan `ID` kendaraan, dimensi dan atribut yang valid, batas geladak, larangan tumpang tindih, serta kapasitas berat kapal.
Neighbor generator	`generate_candidates()`, `get_feasible_neighbors()`	Menghasilkan kandidat melalui pertukaran penempatan (*swap*), pemindahan atau pengeluaran kendaraan (*move*), dan perubahan orientasi (*rotate*). Kandidat yang tidak valid dan duplikat tidak dimasukkan ke daftar tetangga yang layak.
Visualisasi	`plot_ship_states()`, `draw_state()`	Memvisualisasikan konfigurasi awal dan akhir pada geladak kapal, termasuk posisi kendaraan, nilai objektif, total berat, kendaraan yang tidak dimuat, dan informasi validasi.

Kelas `State` menyimpan konfigurasi solusi dalam bentuk dictionary `placements`. Setiap kunci merupakan ID kendaraan, sedangkan nilainya berupa `Placement` atau `None`. Struktur ini memungkinkan algoritma mengubah penempatan kendaraan tanpa mengubah identitas kendaraan yang sedang diproses. Potongan kode representasi state adalah sebagai berikut:

Representasi State

```
@dataclass
class State:
    placements: dict[str, Placement | None] = field(
        default_factory=dict
    )

    def copy(self) -> "State":
        return State(placements=self.placements.copy())

    def loaded_vehicle_ids(self) -> list[str]:
        return [
            vehicle_id
            for vehicle_id, placement in self.placements.items()
            if placement is not None
```

```
        ]

    def unloaded_vehicle_ids(self) -> list[str]:
        return [
            vehicle_id
            for vehicle_id, placement in self.placements.items()
            if placement is None
        ]
```

Metode `copy()` menghasilkan state baru dengan dictionary penempatan yang disalin. Sementara itu, `loaded_vehicle_ids()` dan `unloaded_vehicle_ids()` memisahkan kendaraan yang dimuat dan tidak dimuat berdasarkan nilai penempatannya.

Fungsi `generate_random_state()` pada `src/core/initializer.py` membentuk solusi awal dengan menginisialisasi seluruh kendaraan sebagai belum dimuat. Urutan kendaraan kemudian diacak dan setiap kendaraan dicoba pada berbagai posisi serta orientasi. Penempatan hanya diterima jika menghasilkan state yang valid.

Validasi utama dilakukan oleh `is_valid_state()` pada `src/core/constraints.py`. Fungsi ini memastikan bahwa semua ID kendaraan tercakup, dimensi dan atribut kendaraan valid, setiap penempatan berada di dalam batas kapal, tidak ada kendaraan yang saling tumpang tindih, serta total berat kendaraan tidak melebihi kapasitas kapal.

Potongan kode pemeriksaan constraint adalah sebagai berikut:

Pemeriksaan Constraint

```
def is_within_bounds(
    placement: Placement,
    vehicle: Vehicle,
    ship: Ship,
) -> bool:
    width, length = placement.dimension(vehicle)
    return (
        placement.x >= 0
        and placement.y >= 0
        and placement.x + width <= ship.width
        and placement.y + length <= ship.length
    )
```

Fungsi tersebut memeriksa apakah kendaraan berada sepenuhnya di dalam geladak kapal. Pemeriksaan tumpang tindih dilakukan melalui `has_no_overlap()`, sedangkan

kapasitas berat diperiksa melalui `is_within_capacity()`. Ketiganya digunakan oleh `is_valid_state()` untuk menentukan apakah suatu solusi layak digunakan.

Fungsi `generate_candidates()` menghasilkan kandidat solusi melalui tiga mekanisme. Operasi *swap* menukar penempatan dua kendaraan, *move* memindahkan kendaraan ke posisi lain atau mengeluarkannya dari kapal, dan *rotate* mengubah orientasi kendaraan. Selanjutnya, `get_feasible_neighbors()` memeriksa setiap kandidat, membuang kandidat yang melanggar constraint, dan menghindari duplikasi state. Dengan mekanisme tersebut, algoritma *Local Search* dapat menjelajahi konfigurasi penempatan yang berbeda tanpa menggunakan solusi yang melanggar batasan utama.

Visualisasi diimplementasikan dalam `src/visualization/ship_plot.py` melalui fungsi `plot_ship_states()` dan `draw_state()`. Visualisasi membandingkan konfigurasi awal dan akhir dengan menggambarkan setiap kendaraan sebagai persegi panjang pada geladak kapal. Panel informasi juga menampilkan nilai objektif, total berat, jumlah kendaraan yang dimuat, kendaraan yang tidak dimuat, serta *shipping fee* yang tidak terangkut. Visualisasi ini membantu mengevaluasi perubahan susunan kendaraan dan memeriksa apakah solusi akhir memenuhi batasan penempatan.

2.1. *Steepest-Ascent Hill Climbing*

Algoritma Steepest-Ascent Hill Climbingdimulai dari solusi awal yang memenuhi batasan, kemudian mengevaluasi seluruh *feasible neighbor* dan memilih kandidat dengan nilai objektif tertinggi. Proses berulang selama ditemukan solusi yang lebih baik. Jika tidak ada perbaikan, algoritma berhenti pada local optimum, yang belum tentu merupakan solusi optimal global.

Algoritma Steepest-Ascent Hill Climbing diimplementasikan melalui fungsi `run_hill_climbing()` pada file `src/algorithms/hill_climbing.py`. Fungsi ini menerima solusi awal, data kendaraan, informasi kapal, batas iterasi opsional, dan fungsi objektif. Hasil pencarian dikembalikan dalam bentuk `AlgorithmResult`, yang menyimpan solusi awal dan akhir, solusi terbaik, nilai objektif, runtime, jumlah iterasi, riwayat objektif, serta metadata terminasi.

Signature

```
def run_hill_climbing(
    initial_state: State,
    vehicles: dict[str, Vehicle],
    ship: Ship,
    max_iterations: int | None = None,
    objective=calculate_objective,
) -> AlgorithmResult:
```

Parameter `max_iterations` secara default bernilai `None`, sehingga pencarian berlanjut sampai kondisi terminasi terpenuhi. Jika diberikan nilai tertentu, pencarian dibatasi oleh jumlah iterasi tersebut.

Pembangkitan kandidat solusi dilakukan oleh `generate_candidates()` dan `get_feasible_neighbors()` pada `src/core/neighbor.py`. Kandidat dihasilkan melalui pertukaran posisi dua kendaraan, pemindahan atau pengeluaran kendaraan dari kapal, penempatan kendaraan pada posisi lain, serta perubahan orientasi horizontal dan vertikal. Setiap kandidat diperiksa menggunakan `is_valid_state()` pada `src/core/constraints.py` untuk memastikan bahwa penempatan memenuhi batas geladak, tidak saling tumpang tindih, dan tidak melampaui kapasitas berat kapal. Kandidat yang tidak valid dibuang, sedangkan kandidat duplikat dihindari.

Steepest-Ascent Hill Climbing mengevaluasi seluruh feasible *neighbor* menggunakan fungsi objektif untuk menghitung total *ShippingFee*. Seluruh kandidat perlu dievaluasi agar algoritma dapat memilih tetangga dengan nilai objektif tertinggi, bukan sekadar kandidat pertama yang lebih baik. Jika nilai objektif kandidat terbaik melebihi nilai solusi saat ini, algoritma berpindah ke kandidat tersebut. Jika tidak, pencarian dihentikan karena tidak ditemukan perbaikan. Potongan kode berikut menunjukkan proses evaluasi dan pemilihan tetangga terbaik sesuai implementasi:

Proses Evaluasi

```
neighbor_scores = [
    objective(neighbor, vehicles)
    for neighbor in neighbors
]
best_index = max(
    range(len(neighbors)),
    key=lambda index: neighbor_scores[index],
)
best_neighbor = neighbors[best_index]
neighbor_score = neighbor_scores[best_index]
```

Algoritma memiliki tiga kondisi terminasi utama. Kondisi tersebut dicatat dalam metadata hasil melalui `termination_reason` dan `stop_reason`.

Tabel 2.1.a: Tabel Kondisi Terminasi

stop_reason	Deskripsi
`local_optimum`	Tidak ditemukan tetangga dengan nilai objektif lebih tinggi.
`no_feasible_neighbors`	Tidak tersedia tetangga yang memenuhi batasan.
`max_iterations`	Jumlah iterasi mencapai batas maksimum yang ditentukan.

Batas iterasi ditentukan melalui parameter `max_iterations`. Apabila parameter tersebut tidak diberikan, nilainya adalah None dan pencarian tidak dibatasi oleh jumlah iterasi tertentu. Pencarian tetap dapat berhenti ketika mencapai local optimum atau tidak memiliki tetangga yang valid.

Variabel `history` menyimpan nilai objektif awal dan perkembangan nilai objektif selama pencarian. Pada awal algoritma, nilai objektif solusi awal dimasukkan ke dalam daftar. Setelah setiap iterasi selesai, nilai objektif solusi yang dipertahankan ditambahkan ke dalam history, termasuk ketika algoritma berhenti karena *local optimum*. Riwayat ini digunakan untuk membuat grafik konvergensi dan mengamati perubahan kualitas solusi selama proses pencarian.

Kode berikut menunjukkan bagian utama loop Steepest-Ascent Hill Climbing, mulai dari pembangkitan tetangga, evaluasi seluruh kandidat, pemilihan kandidat terbaik, hingga pemeriksaan terminasi dan pembaruan solusi.

Code Utama Hill Climbing Steepest Ascent

```
while max_iterations is None or iterations < max_iterations:
    neighbors = get_feasible_neighbors(
        current_state,
        vehicles,
        ship,
    )
    if not neighbors:
        termination_reason = "no_feasible_neighbors"
        break

    neighbor_scores = [
```

```
        objective(neighbor, vehicles)
        for neighbor in neighbors
    ]
    best_index = max(
        range(len(neighbors)),
        key=lambda index: neighbor_scores[index],
    )
    best_neighbor = neighbors[best_index]
    neighbor_score = neighbor_scores[best_index]
    iterations += 1

    if neighbor_score <= current_score:
        history.append(current_score)
        termination_reason = "local_optimum"
        break

    current_state = best_neighbor.copy()
    current_score = neighbor_score

    if current_score > best_score:
        best_state = current_state.copy()
        best_score = current_score

    history.append(current_score)
```

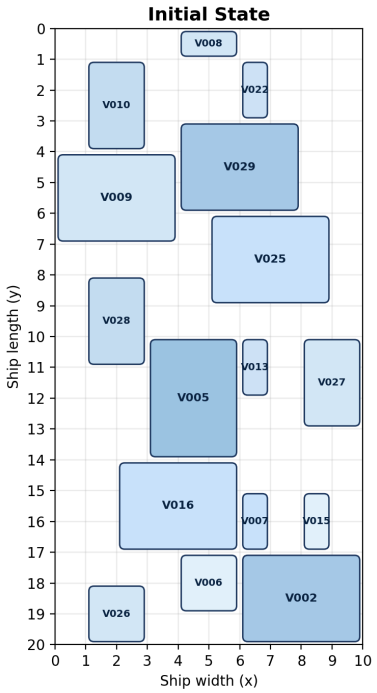
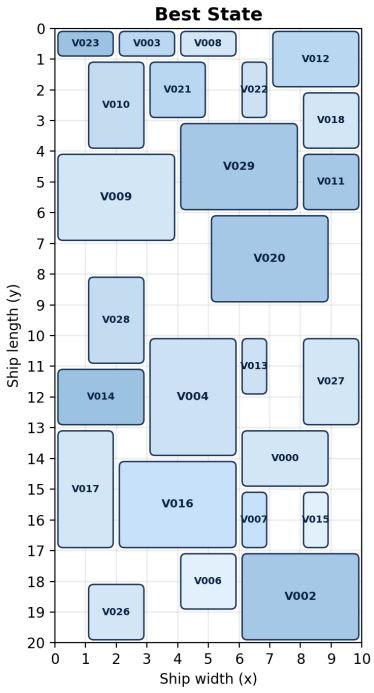
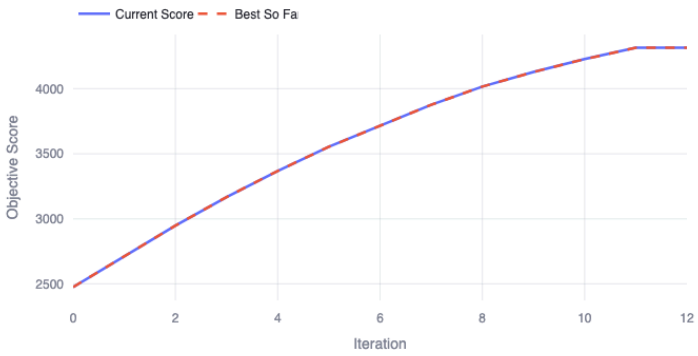
Setelah loop berakhir, algoritma menghitung waktu eksekusi dan mengembalikan hasil melalui `AlgorithmResult`. Hasil tersebut mencakup solusi akhir, solusi terbaik, skor terbaik, jumlah iterasi, `history`, serta metadata yang menjelaskan konfigurasi dan alasan terminasi.

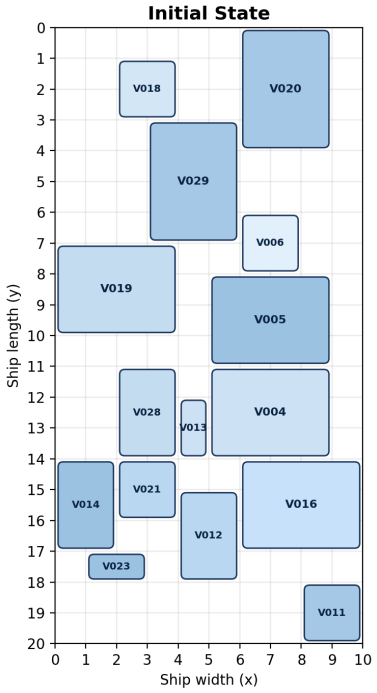
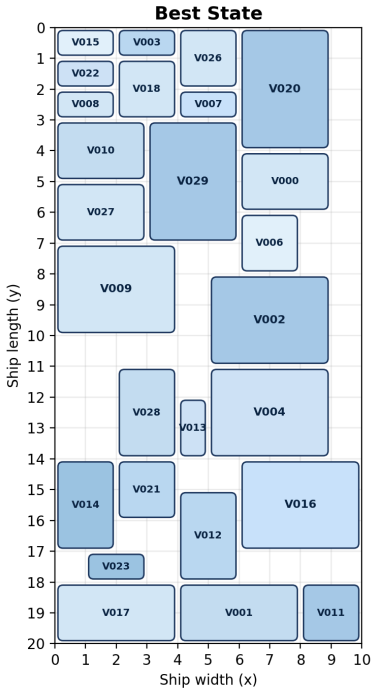
Tabel 2.1.b: Hasil Steepest *Hill Climbing*

Trial	***Seed***	***Initial***	***Final/Best***	***Runtime(s)***	**Iterasi**	***Stop Reason***
1	42	2476	4315	8,133	12	local_optimum
2	43	2477	4385	5,588	14	local_optimum
3	44	2933	4385	10,877	10	local_optimum
Mean			**4361,67**	**8,199**	**12,0**	

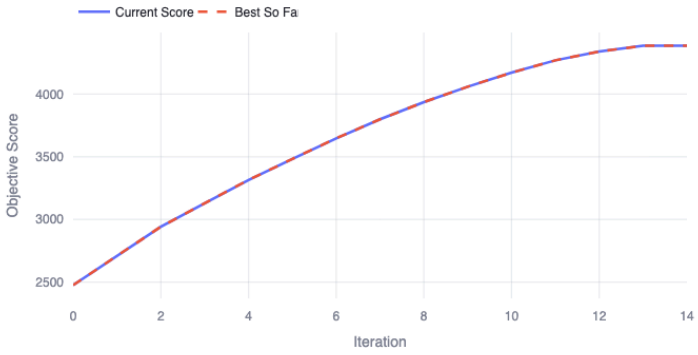
Sumber: output/summary.csv

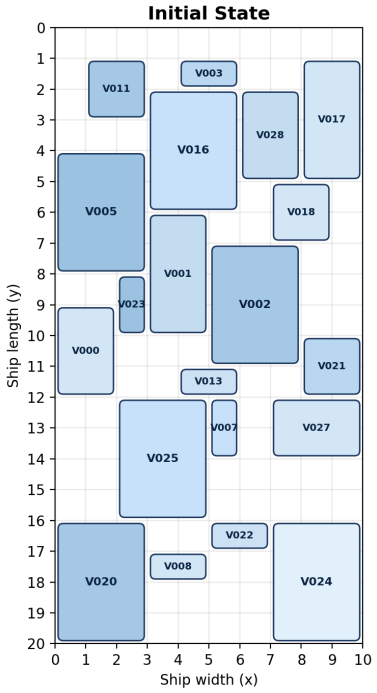
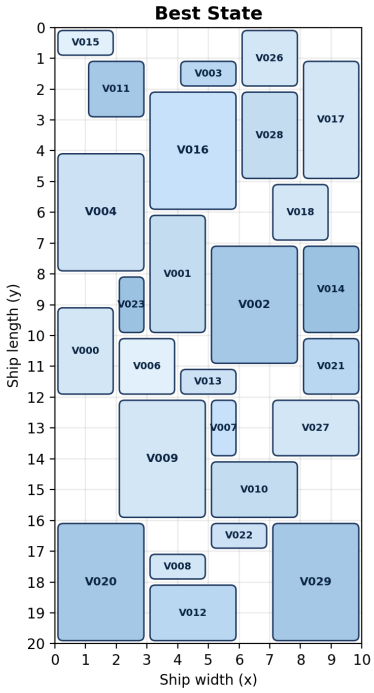
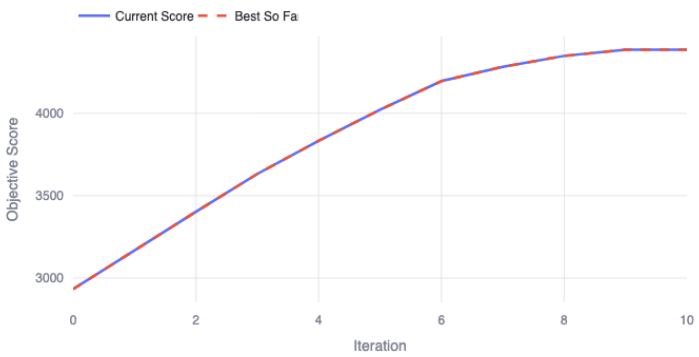
Tabel 2.1.c: Visualisasi Hasil *Steepest Hill Climbing*

Trial	*Initial State*	*Best State*
1		
	Optimization Progress	
		

Trial	Initial State	Best State
2		

Optimization Progress



Trial	*Initial State*	*Best State*
3		
	Optimization Progress	
		

Sample *standard deviation* untuk *best score* adalah 40,41. Ketiga *trial* berhenti karena local_optimum, jauh sebelum batas 1.000 iterasi. Hasil tersebut konsisten dengan karakter *Hill Climbing* yang hanya menerima peningkatan dan karena itu dapat cepat mencapai optimum lokal.

2.2. *Sideways Move Hill Climbing*

Sideways Move Hill Climbing merupakan variasi Steepest-Ascent Hill Climbing yang mengizinkan perpindahan menuju solusi tetangga (*neighbor*) dengan nilai objektif

yang sama dengan solusi saat ini. Mekanisme ini bertujuan membantu pencarian melewati kondisi *plateau*, yaitu keadaan ketika beberapa solusi memiliki nilai objektif yang sama.

Dalam permasalahan pemuatan kendaraan pada geladak kapal, algoritma mengevaluasi seluruh *feasible neighbor* yang dihasilkan melalui operasi `move`, `swap`, dan `rotate`. Algoritma memilih kandidat dengan nilai objektif tertinggi. Jika skor kandidat lebih tinggi, pencarian berpindah ke kandidat tersebut. Jika skor tertingginya sama dengan skor saat ini, perpindahan *sideways* dapat dilakukan selama belum mencapai batas yang ditentukan. Implementasi Sideways Move Hill Climbing menggunakan fungsi `run_hill_climbing_sideways()` dengan parameter utama berikut.

Tabel 2.2.a: Tabel Parameter Sideways Move Hill Climbing

Parameter	Nilai	Deskripsi
`initial_state`	-	Solusi awal yang memenuhi batasan.
`max_sideways`	20	Jumlah maksimum perpindahan dengan nilai objektif yang sama.
`max_iterations`	1000	Batas maksimum iterasi pencarian.
`seed`	None	Seed untuk mengatur pemilihan tetangga secara acak.
`objective`	calculate_objective	Fungsi untuk menghitung total ShippingFee.

Fungsi `get_feasible_neighbors()` digunakan untuk menghasilkan kandidat yang memenuhi kendala, sedangkan `is_valid_state()` memastikan bahwa kondisi awal valid.

Pada setiap iterasi, algoritma menghasilkan seluruh tetangga yang memenuhi kendala, kemudian menghitung nilai objektif masing-masing kandidat. Kandidat dengan skor tertinggi dipilih sebagai tujuan perpindahan.

Jika skor tertinggi lebih rendah daripada skor saat ini, algoritma berhenti karena telah mencapai local optimum. Jika skor tertinggi sama dengan skor saat ini, algoritma menambah penghitung perpindahan *sideways*. Apabila batas perpindahan telah

tercapai, pencarian dihentikan. Jika tidak, algoritma memilih secara acak salah satu kandidat yang memiliki skor tertinggi tersebut.

Potongan kode pemilihan tetangga adalah sebagai berikut:

Pemilihan neighbor Sideways Move

```
scores = [objective(n, vehicles) for n in neighbors]
top = max(scores)

if top < score:
    history.append(score)
    reason = "local_optimum"
    break

if top == score:
    if sideways >= max_sideways:
        history.append(score)
        reason = "max_sideways"
        break
    sideways += 1

current = rng.choice(
    [n for n, s in zip(neighbors, scores) if s == top]
)
score = top
```

Tabel 2.2.b: Tabel Kondisi Terminasi

stop_reason	Deskripsi
`local_optimum`	Seluruh tetangga memiliki nilai objektif lebih rendah daripada solusi saat ini.
`no_feasible_neighbors`	Tidak tersedia tetangga yang memenuhi batasan.
`max_iterations`	Jumlah iterasi mencapai batas maksimum.
`max_sideways`	Batas perpindahan dengan nilai objektif yang sama telah tercapai.

Variabel history menyimpan nilai objektif awal dan nilai solusi setelah setiap iterasi. Nilai yang sama dapat muncul berulang ketika algoritma melakukan *sideways* move. Riwayat ini digunakan untuk menganalisis perkembangan solusi dan mengidentifikasi area datar pada grafik konvergensi.

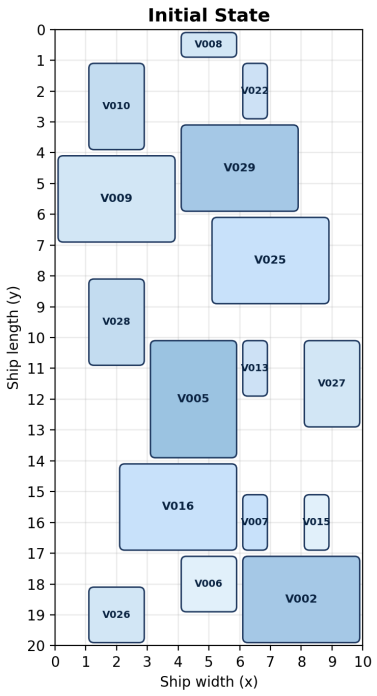
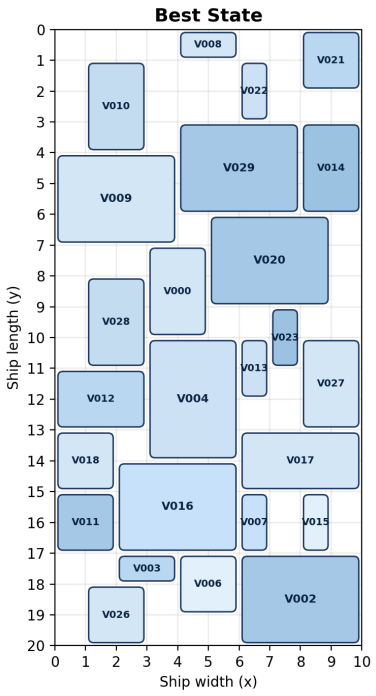
Tabel 2.2.c: Hasil *Sideways Move Hill Climbing*

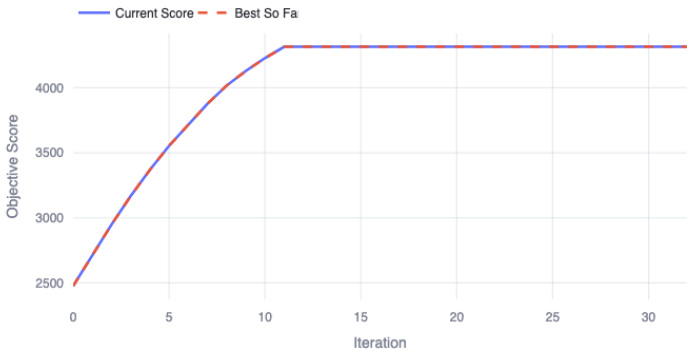
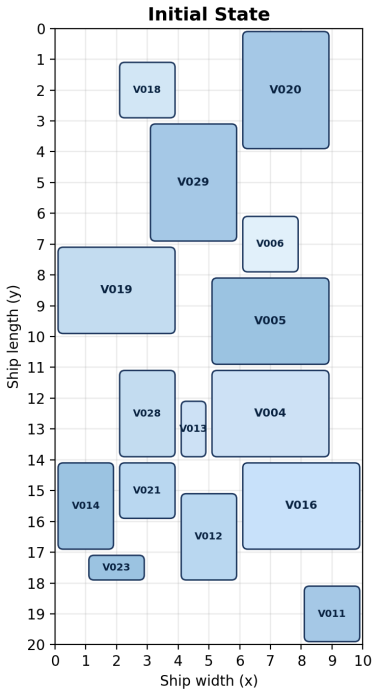
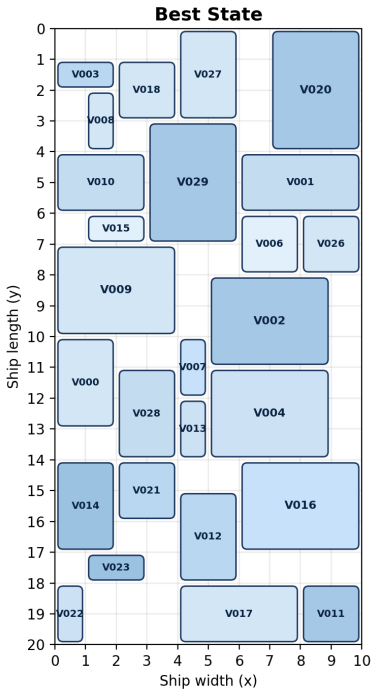
Trial	*Seed*	*Initial*	*Final/Best*	*Runtime(s)*	Iterasi	*Stop Reason*
1	42	2476	4315	11.353	32	max_sideways
2	43	2477	4385	12.477	34	max_sideways
3	44	2933	4385	10.763	30	max_sideways
Mean			**4361.67**	**11.531**	**32**	

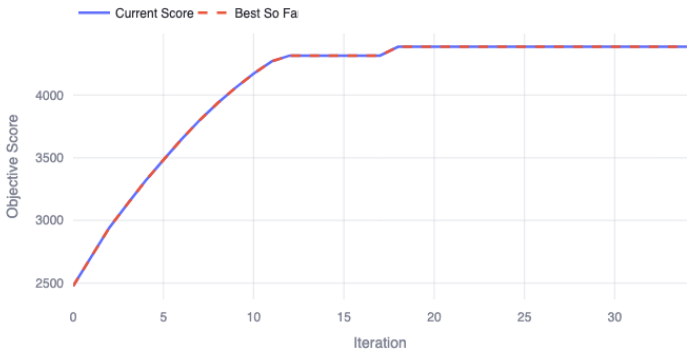
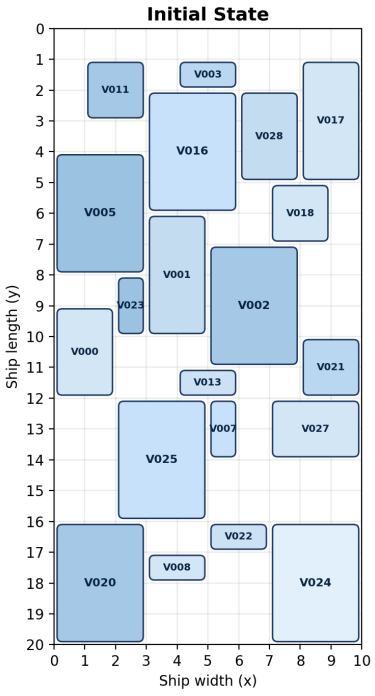
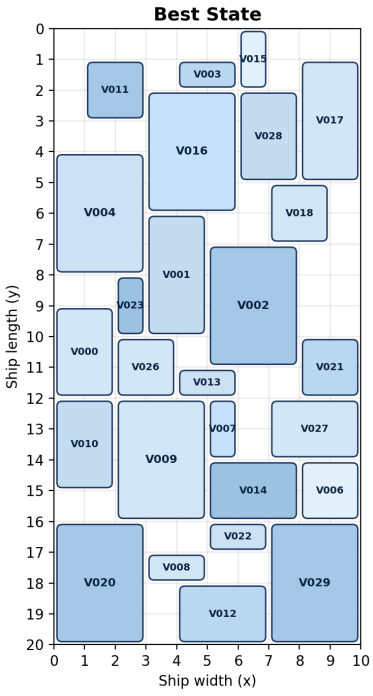
Sumber: output/summary.csv

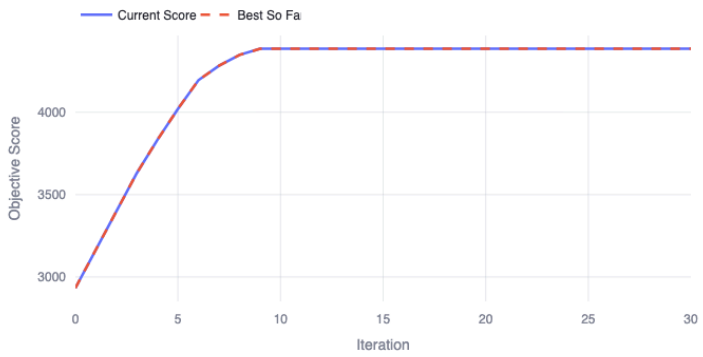
Berdasarkan hasil tersebut, Sideways Move Hill Climbing menghasilkan skor akhir rata-rata sebesar 4.361,67 dengan waktu eksekusi rata-rata 15,451 detik. Seluruh percobaan berhenti karena mencapai batas 20 perpindahan *sideways*. Hasil ini menunjukkan bahwa algoritma dapat menjelajahi solusi dengan skor yang sama, tetapi mekanisme tersebut tidak selalu meningkatkan nilai objektif.

Tabel 2.2.d: Visualisasi Hasil *Sideways Move Hill Climbing*

Trial	*Initial State*	*Best State*
1		
	Optimization Progress	

Trial	Initial State	Best State
		
2		
	Optimization Progress	

Trial	Initial State	Best State
		
3		
	Optimization Progress	

Trial	*Initial State*	*Best State*
		

2.3. *Stochastic Hill Climbing*

Stochastic Hill Climbing merupakan algoritma pencarian lokal yang memilih secara acak satu tetangga dengan nilai objektif lebih tinggi daripada solusi saat ini. Berbeda dengan Steepest-Ascent Hill Climbing yang memilih kandidat dengan skor tertinggi, Stochastic Hill Climbing memilih salah satu kandidat yang memenuhi syarat peningkatan.

Dalam permasalahan pemuatan kendaraan, algoritma mengevaluasi tetangga yang memenuhi kendala geladak kapal, kemudian mengumpulkan kandidat yang meningkatkan total *ShippingFee*. Salah satu kandidat tersebut dipilih secara acak sebagai solusi berikutnya. Pencarian berhenti ketika tidak ditemukan kandidat yang meningkatkan nilai objektif atau ketika batas iterasi tercapai. Implementasi Stochastic Hill Climbing menggunakan fungsi `run_hill_climbing_stochastic()`.

Tabel 2.3.a: Tabel Parameter *Stochastic Hill climbing*

Parameter	Nilai	Deskripsi
`initial_state`	-	Solusi awal yang memenuhi batasan.
`max_iterations`	1000	Batas maksimum iterasi pencarian.
`seed`	None	Seed untuk mengatur pemilihan tetangga secara acak.
`objective`	calculate_objecti ve	Fungsi untuk menghitung total *ShippingFee*.

Fungsi `get_feasible_neighbors()` menghasilkan tetangga yang memenuhi kendala, sedangkan `rng.choice()` digunakan untuk memilih kandidat peningkatan secara acak.

Pada setiap iterasi, algoritma membangkitkan tetangga yang memenuhi kendala dan menghitung nilai objektifnya. Selanjutnya, kandidat yang mempunyai skor lebih tinggi daripada skor saat ini dimasukkan ke dalam daftar improving.

Jika daftar tersebut tidak kosong, algoritma memilih salah satu kandidat secara acak dan memperbarui solusi saat ini. Jika tidak terdapat kandidat peningkatan, algoritma menghentikan pencarian. Kondisi ini dicatat sebagai `local_optimum` apabila tetangga tersedia, atau `no_feasible_neighbors` apabila tidak terdapat tetangga yang memenuhi kendala.

Potongan kode implementasinya adalah sebagai berikut:

Pemilihan neighbor Stochastic Hill Climbing

```
neighbors = get_feasible_neighbors(
    current, vehicles, ship
)

improving = [
    (n, objective(n, vehicles))
    for n in neighbors
    if objective(n, vehicles) > score
]

iterations += 1

if not improving:
    history.append(score)
    reason = (
        "local_optimum"
        if neighbors
        else "no_feasible_neighbors"
    )
    break

current, score = rng.choice(improving)

if score > best_score:
    best, best_score = current.copy(), score

history.append(score)
```

Tabel 2.3.b: Tabel Kondisi Terminasi

stop_reason	Deskripsi
`local_optimum`	Terdapat tetangga yang valid, tetapi tidak ada yang meningkatkan nilai objektif.
`no_feasible_neighbors`	Tidak tersedia tetangga yang memenuhi batasan.
`max_iterations`	Jumlah iterasi mencapai batas maksimum.

Variabel history menyimpan nilai objektif awal dan nilai solusi setelah setiap iterasi. Karena algoritma hanya menerima tetangga yang meningkatkan nilai objektif, nilai dalam riwayat tidak menurun. Apabila tidak ditemukan kandidat yang lebih baik, nilai saat ini dicatat sebelum algoritma berhenti.

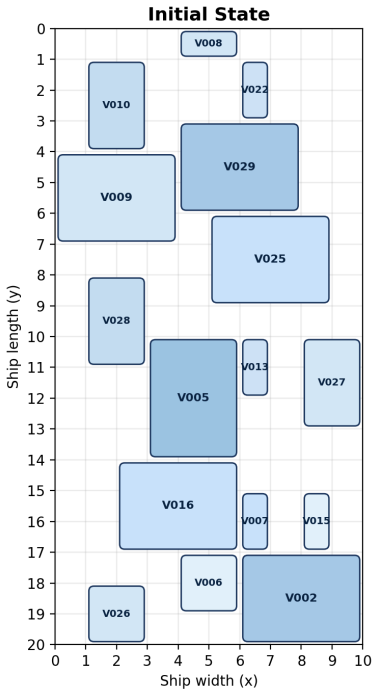
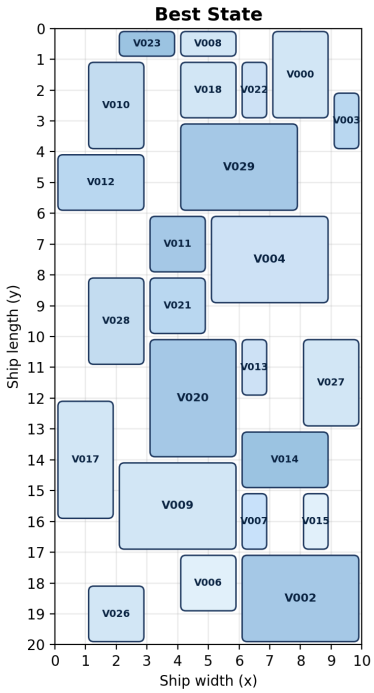
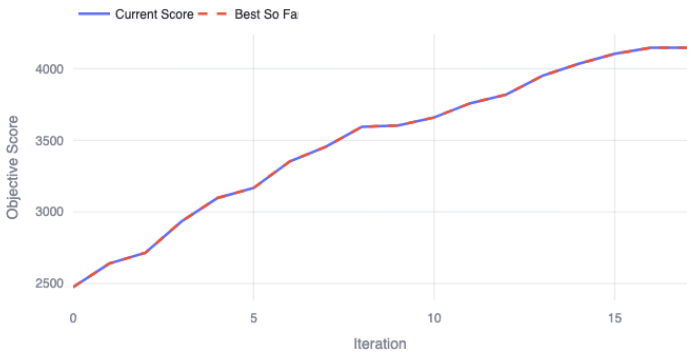
Tabel 2.3.c: Hasil *Stochastic Hill Climbing*

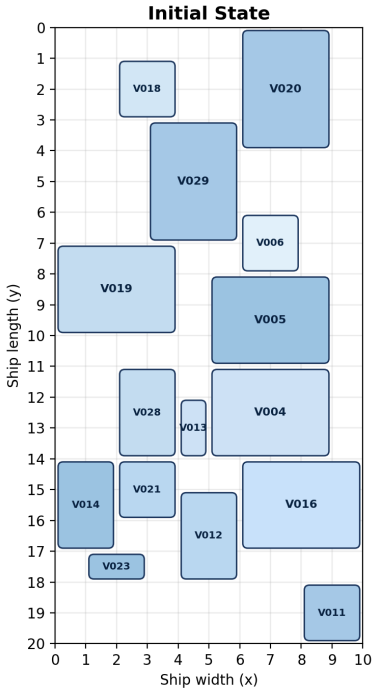
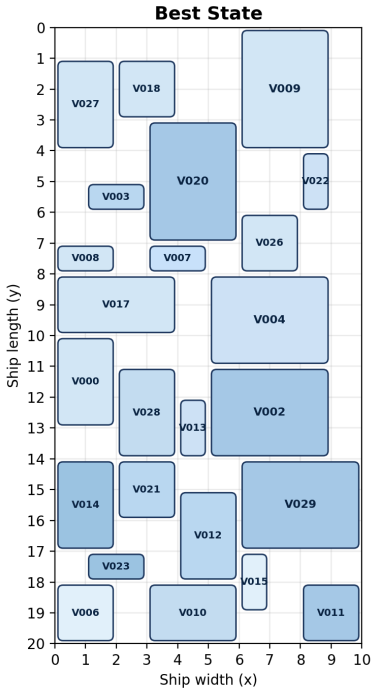
Trial	*Seed*	*Initial*	*Final/Best*	*Runtime(s)*	Iterasi	*Stop Reason*
1	42	2476	4146	6.043	17	local_optimum
2	43	2477	4146	6.727	19	local_optimum
3	44	2933	4045	4.967	14	local_optimum
Mean			**4112.33**	**5.912**	**16.67**	

Sumber: output/summary.csv

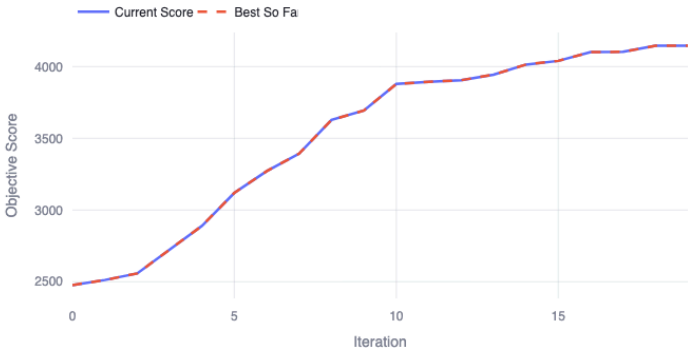
Stochastic Hill Climbing memperoleh skor akhir rata-rata sebesar 4.112,33 dan waktu eksekusi rata-rata 5.912 detik. Ketiga percobaan berhenti karena tidak ditemukan tetangga yang meningkatkan skor. Dibandingkan dengan Sideways Move Hill Climbing, algoritma ini memiliki waktu eksekusi rata-rata yang lebih rendah, tetapi menghasilkan skor akhir rata-rata yang juga lebih rendah. Hal tersebut menunjukkan bahwa pemilihan kandidat secara acak tidak selalu menghasilkan solusi dengan nilai objektif tertinggi.

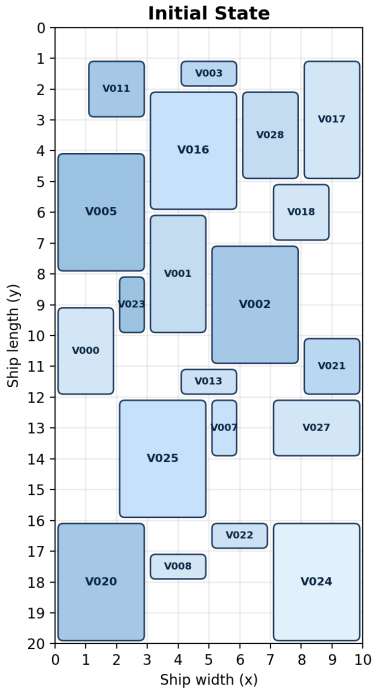
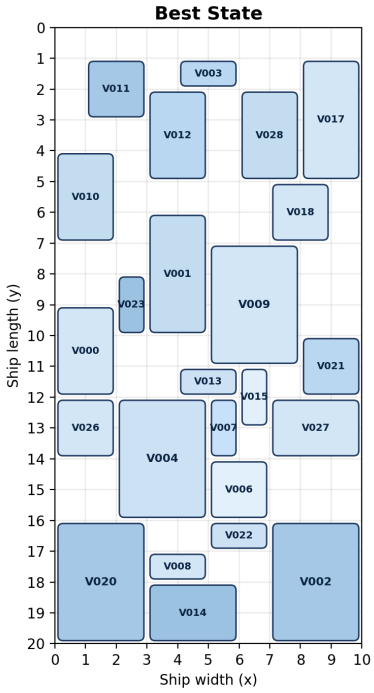
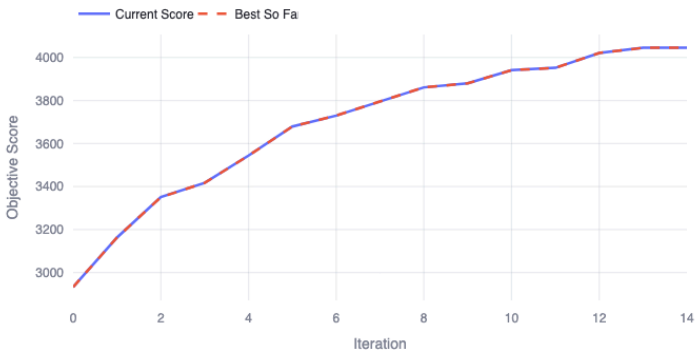
Tabel 2.3.d: Visualisasi Hasil *Stochastic Hill Climbing*

Trial	*Initial State*	*Best State*
1		
	Optimization Progress	
		

Trial	Initial State	Best State
2		

Optimization Progress



Trial	Initial State	Best State
3		
	Optimization Progress	
		

2.4. *Random Restart Hill Climbing*

Random Restart Hill Climbing merupakan variasi Hill Climbing yang menjalankan pencarian berulang kali dari beberapa solusi awal. Pendekatan ini bertujuan mengurangi ketergantungan terhadap satu solusi awal dan meningkatkan peluang menemukan solusi dengan nilai objektif yang lebih baik.

Pada implementasinya, percobaan pertama menggunakan `initial_state` yang diberikan. Percobaan berikutnya menggunakan solusi awal baru yang dibangkitkan melalui `generate_random_state()`. Setiap solusi awal diproses menggunakan Steepest-Ascent Hill Climbing, kemudian skor terbaik dari seluruh percobaan dibandingkan untuk menentukan solusi akhir. Implementasi Random Restart Hill Climbing menggunakan fungsi `run_hill_climbing_random_restart()`.

Tabel 2.4.a: Tabel Parameter *Random Restart Hill Climbing*

Parameter	Nilai	Deskripsi
`initial_state`	-	Solusi awal yang memenuhi batasan.
`max_iterations`	1000	Batas maksimum iterasi pencarian.
`max_restarts`	5	Jumlah restart tambahan setelah percobaan pertama.
`load_prob`	0.5	Peluang kendaraan dicoba untuk dimuat saat membangkitkan solusi awal baru.
`seed`	None	Seed untuk menghasilkan solusi awal yang dapat direproduksi.
`objective`	calculate_objective	Fungsi untuk menghitung total ShippingFee.

Fungsi `generate_random_state()` menghasilkan solusi awal untuk restart, sedangkan `run_hill_climbing()` menjalankan pencarian lokal dari setiap solusi awal tersebut.

Pada setiap restart, algoritma menjalankan Steepest-Ascent Hill Climbing yang mengevaluasi seluruh tetangga layak dan memilih kandidat dengan skor tertinggi. Pencarian pada satu solusi awal berhenti ketika mencapai kondisi terminasi Hill Climbing.

Setelah pencarian selesai, algoritma menyimpan skor terbaik dan jumlah iterasinya. Proses tersebut diulangi hingga seluruh restart selesai. Dengan `max_restarts=5`, algoritma menjalankan enam pencarian: satu pencarian awal dan

lima restart tambahan. Solusi dengan skor terbaik dari seluruh pencarian dipilih sebagai hasil akhir.

Potongan kode berikut menunjukkan proses pembangkitan solusi awal, pelaksanaan pencarian, dan pemilihan solusi terbaik.

Pemilihan neighbor Random Restart Hill Climbing

```
for r in range(max_restarts + 1):
    s0 = (
        initial_state
        if r == 0
        else generate_random_state(
            vehicles,
            ship,
            load_prob=load_prob,
            seed=seed_rng.randrange(2**31),
        )
    )

    res = run_hill_climbing(
        s0, vehicles, ship, max_iterations, objective
    )

    history.extend(res.history)
    per_restart.append(res.iterations)
    scores_per_restart.append(res.best_score)
    iterations += res.iterations

    if res.best_score > best_score:
        best, best_score, best_final = (
            res.best_state,
            res.best_score,
            res.final_state,
        )
```

Random Restart Hill Climbing menghentikan proses setelah seluruh percobaan selesai. Kondisi dan informasi yang dicatat dirangkum pada tabel berikut.

Tabel 2.4.b: Tabel Kondisi Terminasi

stop_reason	Deskripsi
`stop_reason`	Bernilai max_restarts setelah seluruh percobaan selesai.
`max_restarts`	Jumlah restart tambahan yang dilakukan setelah percobaan pertama.
`iterations_per_resta`	Daftar jumlah iterasi pada setiap percobaan.

`rt`	
`best_score_per_restart`	Daftar skor terbaik yang ditemukan pada setiap percobaan.
`history`	Gabungan riwayat nilai objektif dari seluruh percobaan.

Terdapat dua tingkat terminasi pada algoritma ini. Pertama, setiap pencarian Hill Climbing berhenti karena mencapai `local_optimum`, `no_feasible_neighbors`, atau `max_iterations`. Kedua, keseluruhan Random Restart Hill Climbing berhenti setelah jumlah restart yang ditentukan selesai dijalankan. Implementasi mencatat terminasi keseluruhan sebagai max_restarts.

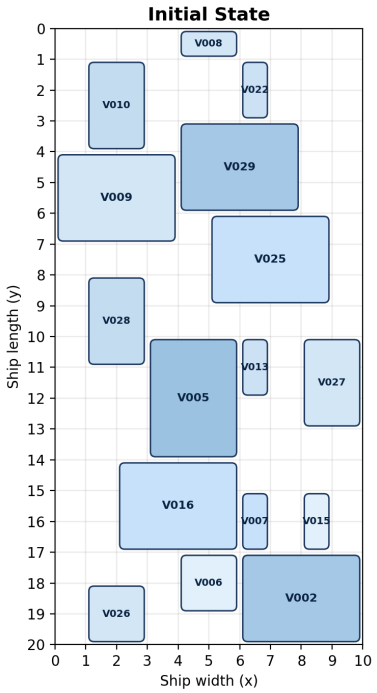
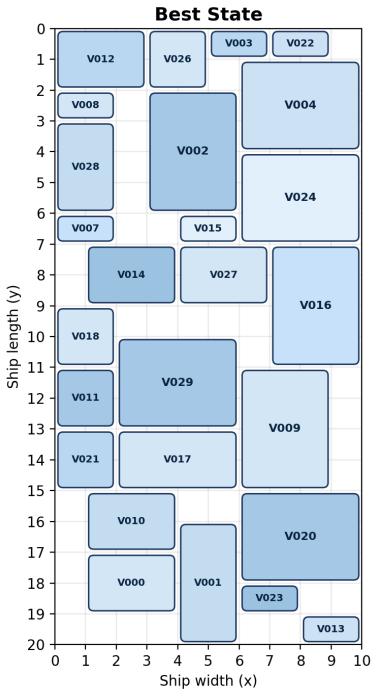
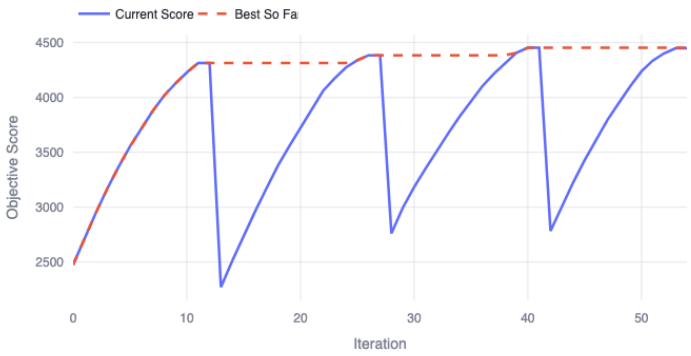
Tabel 2.4.c: Hasil *Random Restart Hill Climbing*

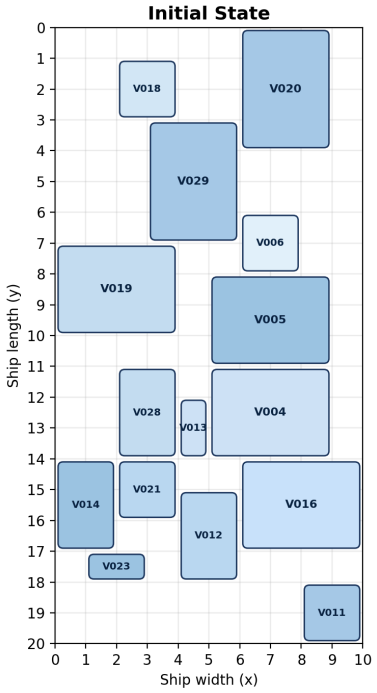
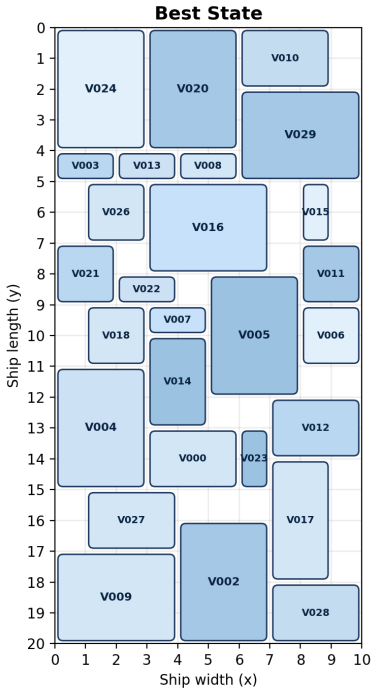
Trial	***Seed***	***Initial***	***Final/Best***	***Runtime(s)***	**Iterasi**	***Stop Reason***
1	42	2476	4453	27.308	51	max_restarts
2	43	2477	4557	25.131	47	max_restarts
3	44	2933	4385	27.210	51	max_restarts
Mean			**4465**	**26.549**	**49.67**	

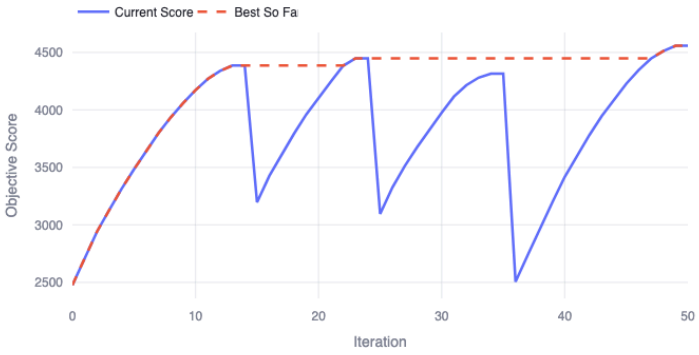
Sumber: output/summary.csv

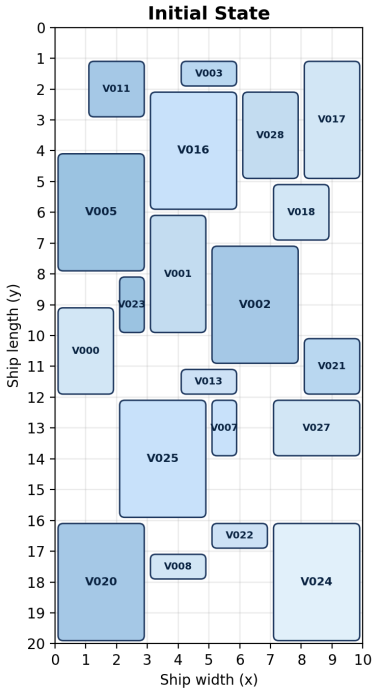
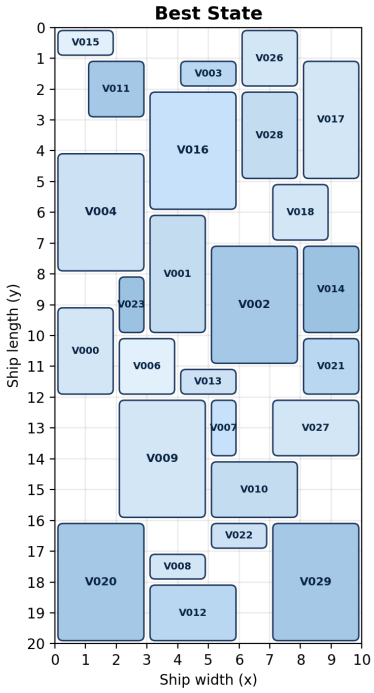
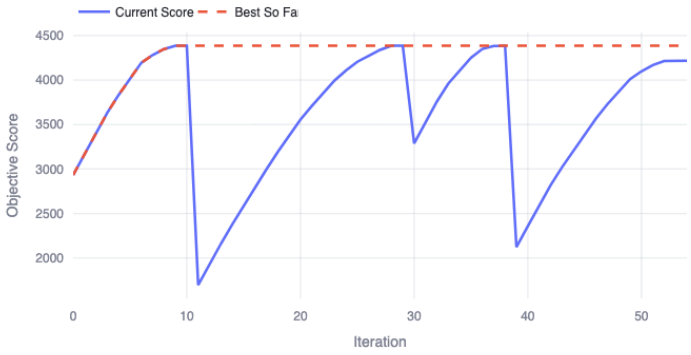
Random Restart Hill Climbing memperoleh skor akhir rata-rata sebesar 4.465, tertinggi di antara ketiga variasi yang dibahas. Hasil ini menunjukkan bahwa pencarian dari beberapa solusi awal dapat meningkatkan peluang menemukan solusi yang lebih baik. Namun, waktu eksekusi rata-ratanya mencapai 49,67 detik karena algoritma menjalankan Hill Climbing berulang kali. Dengan demikian, peningkatan kualitas solusi disertai biaya komputasi yang lebih besar.

Tabel 2.4.d: Visualisasi Hasil *Random Restart Hill Climbing*

Trial	*Initial State*	*Best State*
1		
	Optimization Progress	
		

Trial	Initial State	Best State
2		

Optimization Progress


Trial	Initial State	Best State
3		
	Optimization Progress	
		

2.5. *Simulated Annealing*

Simulated Annealing memulai pencarian dari *initial state feasible* dan menggunakan satu kandidat *neighbor* pada setiap iterasi. *Candidate* dengan *objective* lebih tinggi diterima secara langsung. *Candidate* yang lebih buruk masih dapat diterima dengan probabilitas yang bergantung pada selisih *objective* dan temperatur. Temperatur berkurang menggunakan *cooling schedule* sampai memenuhi kondisi terminasi. Implementasi SA terdapat pada

`src/algorithms/simulated_annealing.py`, melalui fungsi `run_simulated_annealing()`. Parameter utamanya dirangkum pada tabel berikut.

Tabel 2.5.a: Tabel Parameter *Simulated Annealing*

Parameter	Nilai
Initial temperature	1000,0
Cooling state	0,9995
Minimum temperature	0,1
Maximum Iterations	50.000
Stuck-check interval	200

Simulated Annealing menggunakan probabilitas penerimaan *(acceptance probability)* untuk menentukan apakah kandidat solusi diterima. Untuk masalah maksimisasi, perubahan nilai objektif dihitung dengan rumus:

$$\Delta E = f(s') - f(s)$$

dengan (f(s')) sebagai nilai objektif kandidat dan (f(s)) sebagai nilai objektif solusi saat ini. Probabilitas penerimaan mengikuti rumus berikut:

$$P(\text{accept}) = \begin{cases} 1, & \Delta E \geq 0 \\ e^{\Delta E/T}, & \Delta E < 0 \end{cases}$$

dengan (T) sebagai temperatur saat ini. Kandidat yang meningkatkan atau mempertahankan nilai objektif selalu diterima. Sebaliknya, kandidat yang lebih buruk diterima apabila bilangan acak yang dihasilkan lebih kecil daripada probabilitas penerimaannya. Ketika temperatur menurun, kemungkinan menerima solusi yang lebih buruk juga cenderung berkurang. Potongan kode implementasinya adalah sebagai berikut.

Acceptance Probability Code

```
delta_e = cand_score - current_score
exponent = delta_e / temperature
exp_value = exp(min(exponent, 700))

if delta_e >= 0:
    prob, accept = 1.0, True
else:
    prob = exp_value
```

```
    accept = rng.random() < prob
```

Variabel prob menyimpan probabilitas penerimaan, sedangkan `exp_value` menyimpan hasil eksponensial $e^{\Delta E/T}$ yang digunakan dalam visualisasi. Pada implementasi ini, nilai eksponen dibatasi hingga 700 untuk menghindari masalah numerik.

Temperatur dikurangi secara geometris pada setiap iterasi menggunakan rumus:

$$T_{t+1} = T_t \times \alpha$$

dengan T_t sebagai temperatur saat ini dan α sebagai `cooling_rate`. Dengan nilai default 0,995, temperatur turun secara bertahap sehingga pencarian menjadi semakin selektif terhadap solusi yang lebih buruk.

Kondisi terminasi yang digunakan dijelaskan pada tabel berikut.

Tabel 2.5.b: Tabel Kondisi Terminasi

stop_reason	Deskripsi
`min_temperature`	Temperatur mencapai atau turun di bawah batas minimum.
`no_feasible_neighbors`	Tidak tersedia tetangga valid ketika pembangkitan kandidat acak gagal dan pemeriksaan alternatif juga tidak menemukan kandidat.
`max_iterations`	Jumlah iterasi mencapai batas maksimum.

Pada implementasi ini, kondisi `min_temperature` diperiksa di awal setiap iterasi. Oleh karena itu, pencarian dapat berhenti sebelum mencapai `max_iterations`. Kondisi *local optimum* tidak secara langsung menghentikan Simulated Annealing karena algoritma masih dapat menerima solusi yang lebih buruk untuk menjelajahi ruang pencarian.

Untuk mengamati kondisi pencarian yang terjebak pada *local optimum*, algoritma melakukan pemeriksaan setiap `stuck_check_interval` iterasi. Pemeriksaan dilakukan dengan mengevaluasi tetangga yang memenuhi batasan dan

menentukan apakah ada tetangga dengan nilai objektif lebih tinggi daripada solusi saat ini.

Tabel 2.5.c: Tabel Variabel Stuck

stop_reason	Deskripsi
stuck_count	Jumlah pemeriksaan yang menemukan solusi saat ini berada pada *local optimum*.
stuck_episodes	Jumlah rangkaian maksimum pemeriksaan *stuck* yang berurutan.
stuck_checks	Total pemeriksaan kondisi stuck yang telah dilakukan.
stuck_frequency	Rasio stuck_count terhadap `stuck_checks`.

Frekuensi stuck dihitung dengan rumus:

$$\text{stuck_frequency} = \begin{cases} \dfrac{\text{stuck_count}}{\text{stuck_checks}}, & \text{stuck_checks} > 0 \\ 0, & \text{lainnya} \end{cases}$$

Pemeriksaan ini hanya mencatat kondisi lokal pada interval tertentu dan tidak menghentikan algoritma. Dengan demikian, Simulated Annealing tetap dapat melanjutkan pencarian meskipun beberapa pemeriksaan menunjukkan kondisi *local optimum*.

Simulated Annealing menyimpan perkembangan pencarian dalam beberapa riwayat. Variabel `history` menyimpan nilai objektif solusi saat ini, sedangkan `temperature_history` mencatat temperatur pada setiap iterasi. Selain itu, `acceptance_probability_history` menyimpan probabilitas penerimaan dan `exponential_history` menyimpan nilai eksponensial yang digunakan dalam perhitungan probabilitas.

Data tersebut dapat digunakan untuk menghasilkan dua visualisasi utama. Grafik *acceptance probability* memperlihatkan perubahan peluang penerimaan kandidat, khususnya kandidat yang lebih buruk, seiring perubahan temperatur. Sementara itu, grafik *exponential history* memperlihatkan perubahan nilai $e^{\Delta E/T}$ selama pencarian. Untuk kandidat yang lebih baik atau sama baiknya, probabilitas penerimaan ditetapkan

menjadi 1,0, meskipun nilai eksponensial yang dicatat tetap berasal dari perhitungan ΔE.

Berikut potongan loop utama Simulated Annealing yang mencakup pemeriksaan temperatur, pembangkitan kandidat, perhitungan probabilitas penerimaan, pembaruan solusi, serta penyimpanan riwayat.

Code Utama Simulated Annealing

```
while iterations < max_iterations:
    if temperature <= min_temperature:
        reason = "min_temperature"
        break

    candidate = propose_random_neighbor(
        current, vehicles, ship, rng
    )

    if candidate is None:
        all_neighbors = get_feasible_neighbors(
            current, vehicles, ship
        )
        if not all_neighbors:
            reason = "no_feasible_neighbors"
            break
        candidate = rng.choice(all_neighbors)

    cand_score = objective(candidate, vehicles)
    delta_e = cand_score - current_score
    exponent = delta_e / temperature
    exp_value = exp(min(exponent, 700))

    if delta_e >= 0:
        prob, accept = 1.0, True
    else:
        prob = exp_value
        accept = rng.random() < prob

    temperature_history.append(temperature)
    exponential_history.append(exp_value)
    acceptance_probability_history.append(prob)
    accepted_flags.append(accept)

    if accept:
        current, current_score = candidate, cand_score
        if current_score > best_score:
            best, best_score = current.copy(), current_score

    iterations += 1
    history.append(current_score)
    temperature *= cooling_rate
```

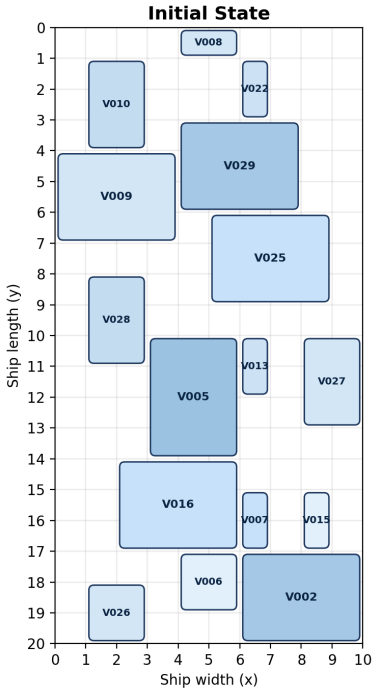
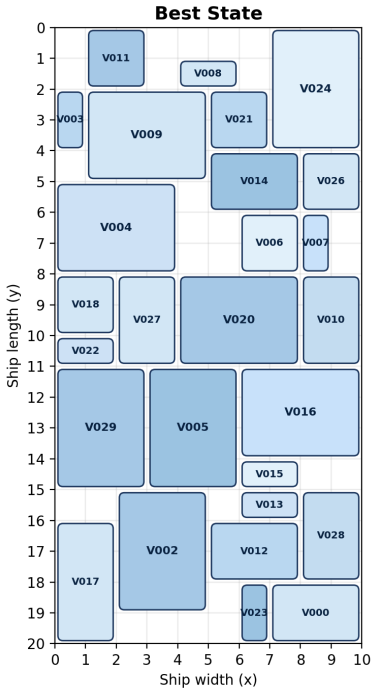
Setelah loop selesai, algoritma mengembalikan `AlgorithmResult` yang berisi solusi awal, solusi akhir, solusi terbaik, skor terbaik, runtime, jumlah iterasi, history, serta metadata temperatur, probabilitas penerimaan, dan statistik *stuck*.

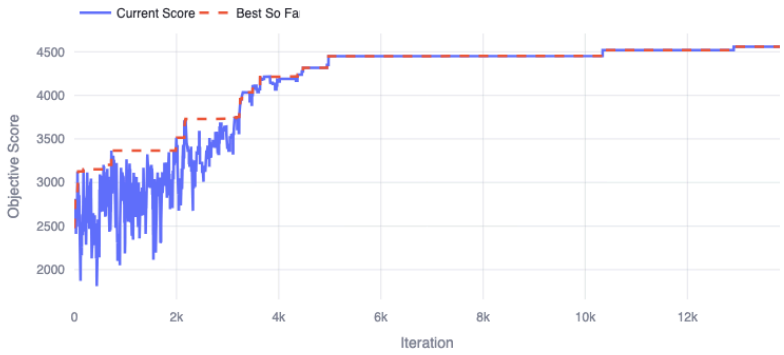
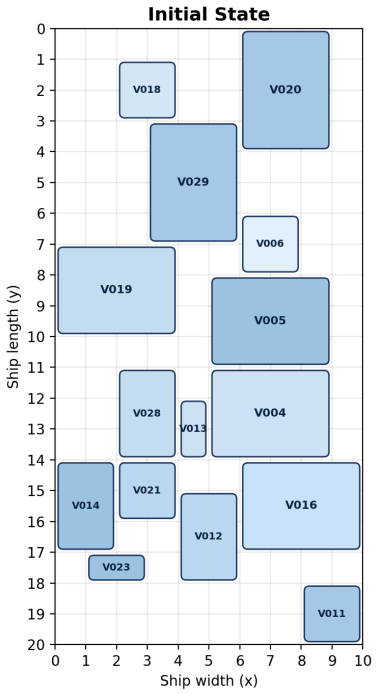
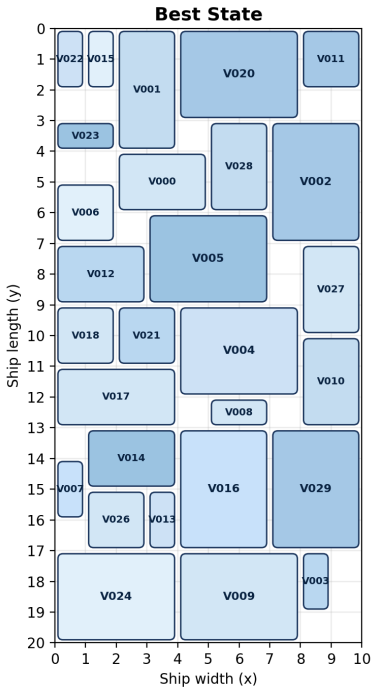
Tabel 2.5.d: Hasil Simulated Annealing (Spesifikasi Wajib)

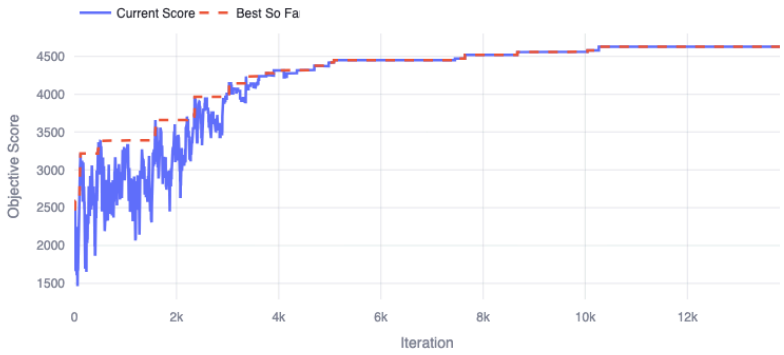
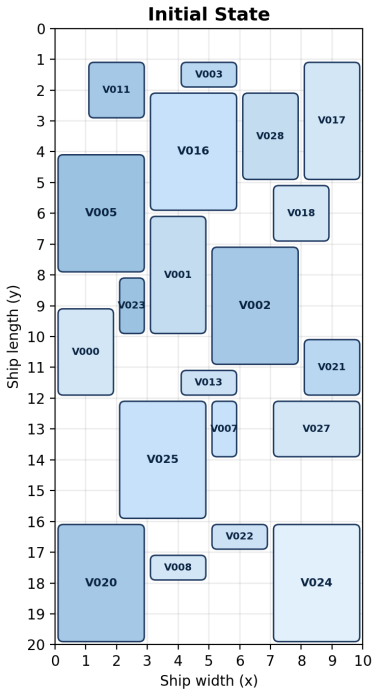
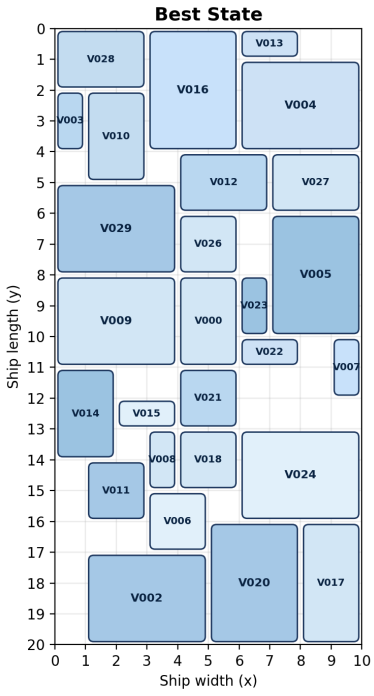
Trial	*Seed*	*Initial*	*Final/Best*	*Runtime(s)*	Iterasi	*Stuck Count*	*Stuck Episodes*
1	42	2476	4557	93,787	18.417	44	3
2	43	2477	4627	39,459	18.417	44	5
3	44	2933	4557	80,743	18.417	42	2
Mean			**4580,33**	**71,330**	**18.417**	**43,33**	**3,33**

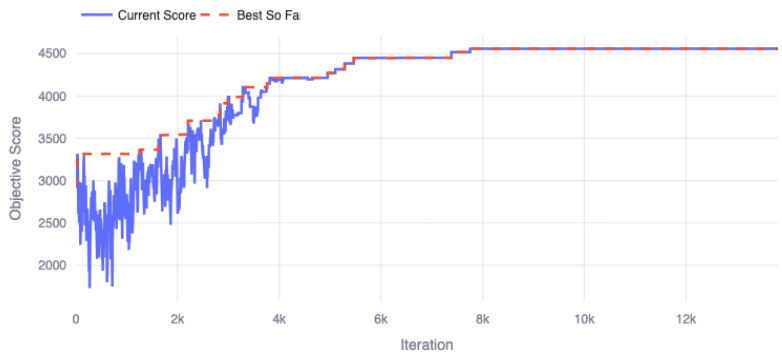
Sumber: output/summary.csv

Tabel 2.5.e: Visualisasi Hasil *Simulated Annealing*

Trial	*Initial State*	*Best State*
1		
	Optimization Progress	

Trial	Initial State	Best State
		
2		
	Optimization Progress	

Trial	Initial State	Best State
		
3		
	Optimization Progress	

Trial	*Initial State*	*Best State*
		

Sample standard deviation best score Simulated Annealing adalah 40,41. Semua *run* berhenti akibat min_temperature. Hasil terbaik individual dari seluruh algoritma wajib adalah 4.627 pada *Simulated Annealing* dengan *seed* 43.

2.6. *Genetic Algorithm*

Genetic Algorithm melakukan pencarian berbasis populasi. Sekumpulan *state* diperlakukan sebagai populasi individu, di mana individu yang lebih baik memperoleh peluang lebih besar menjadi *parent*, lalu *crossover* dan *mutation* menghasilkan generasi berikutnya. Implementasi pada tugas besar ini menggunakan *roulette-wheel selection*, *one-point crossover*, *feasibility repair*, mutation berbasis operasi state, dan *pure generational replacement*. Implementasi juga mencatat maksimum dan rata-rata *fitness* setiap generasi. Jumlah populasi dijadikan kontrol dan jumlah iterasi divariasikan ke tiga nilai. Masing-masing konfigurasi harus dijalankan tiga kali.

Tabel 2.6.a: Tabel Deskripsi Fungsi pada *Genetic Algorithm*

Fungsi	Peran
initialize_population()	Membentuk *feasible population* dengan *initial state* secara eksplisit dimasukkan sebagai individu pertama.
evaluate_population()	Menghitung *fitness* semua individu serta maksimum dan rata-rata.
select_parent_index()	*Roulette-wheel selection* berdasarkan *fitness*.
crossover_one_point()	Menggabungkan *placement* dua *parent* pada satu titik potong.

repair_state()	Memperbaiki *child* yang tidak *feasible* akibat *crossover.*
mutate_state()	Melakukan operasi *move*, *swap*, atau *rotate.*
run_genetic_algorithm()	Mengatur generasi, *tracking fitness*, *global best*, *runtime*, dan *metadata*.

Pertama, *initial state* yang diberikan dimasukkan sebagai individu pertama, sedangkan individu lain dihasilkan secara acak. Seluruh individu diverifikasi dengan `is_valid_state()`. Dengan cara ini, populasi awal mengandung *initial state* yang sama dengan eksperimen algoritma lain sekaligus memperoleh diversitas dari *state* tambahan. *Fitness* suatu individu adalah total *ShippingFee* kendaraan yang dimuat. Karena bobot *selection* harus nonnegatif dan terhingga, masukan *fitness* negatif, *boolean*, atau *nonfinite* ditolak. Fungsi `evaluate_population()` juga menghitung nilai maksimum dan rata-rata populasi. Dua statistik inilah yang dicatat setiap generasi agar eksperimen memenuhi ketentuan *plot Genetic Algorithm*.

Kemudian, *parent* dipilih dengan *roulette-wheel selection*. Probabilitas pemilihan meningkat bersama *fitness*. Jika seluruh *fitness* bernilai nol, implementasi melakukan pemilihan *uniform* secara acak sehingga proses tetap terdefinisi. Jika ada nilai positif, probabilitas individu j dipilih sebanding dengan *fitness*-nya.

Setelah itu, *crossover* diimplementasi menggunakan *one-point crossover*. ID kendaraan diproses dalam urutan yang deterministik, di mana sebagian *placement* berasal dari *parent* A dan sisanya dari *parent* B. Karena kombinasi *placement* dari kedua *parent* dapat menyebabkan *overlap* atau konflik *constraint*, hasil *crossover* tidak langsung diasumsikan *feasible*. Fungsi repair_state() membangun *state* baru dari kondisi kosong. Kendaraan kandidat yang semula dimuat diurutkan berdasarkan `shipping_fee` secara menurun. Posisi warisan dipertahankan apabila masih valid. Jika terjadi konflik, fungsi mencoba relokasi dengan orientasi awal maupun orientasi alternatif. Kendaraan yang melampaui kapasitas atau tidak dapat ditempatkan secara *feasible* dapat dikeluarkan. Heuristik prioritas `shipping_fee` ini secara eksplisit mengarahkan *repair* untuk mempertahankan kendaraan bernilai tinggi.

Selanjutnya, *mutation* memilih salah satu operasi *move, swap*, atau *rotate* berdasarkan kondisi *state*. Operasi *move* juga mempunyai probabilitas 0,2 untuk

mengeluarkan kendaraan yang sedang dimuat. Fungsi mencoba maksimal 32 proposal dan mengembalikan *state* baru hanya jika proposal berbeda serta *feasible*. Bila tidak ditemukan perubahan valid, fungsi mengembalikan *copy state* awal.

Terakhir, seluruh populasi generasi lama digantikan oleh *child* yang baru, sehingga implementasi tidak menggunakan *elitism*. Akan tetapi, `best_state`, `best_score`, dan `best_generation` disimpan secara terpisah. Hal ini penting karena individu terbaik dapat hilang dari populasi selanjutnya.

Genetic Algorithm Pseudocode

```
population ← initialize_population()
evaluate(population)
best_state ← best individual in population

for each generation:
       next_population ← []

       repeat population_size times:
              parent_a ← roulette_selection(child)
              parent_b ← roulette_selection(child)
              child ← one_point_crossover(parent_a, parent_b)
              child ← repair_state(child)

              with probability mutation_rate:
                     child ← mutate_state(child)

              next_population.append(child)

       population ← next_population
       evaluate(population)
       record maximum fitness
       record average fitness

       If generation best > global best:
              update global best

Return final population state, global best, histories
```

Jumlah evaluasi *fitness* yang dicatat untuk satu *run* adalah $N_{eval} = P(G + 1)$ dengan P ukuran populasi dan G jumlah generasi. Untuk konfigurasi 40×100, setiap *trial* mencatat 4040 *fitness evaluations*.

Berikut ini *source code* untuk setiap fungsi dan kelas dari *Genetic Algorithm*:

Population initialization

```
def initialize_population (
```

```
    initial_state: State,
    vehicles: dict[str, Vehicle],
    ship: Ship,
    population_size: int,
    *,
    seed: int | None = None,
) -> list[State]:
    if type(population_size) is not int or population_size < 2:
        raise ValueError("population_size must be an integer >= 2")
    if not is_valid_state(initial_state, vehicles, ship):
        raise ValueError("initial_state must be feasible")

    rng = random.Random(seed)
    population = [initial_state.copy()]
    for _ in range(population_size - 1):
        individual = generate_random_state(
            vehicles, ship, seed=rng.getrandbits(64)
        )
        if not is_valid_state(individual, vehicles, ship):
            raise RuntimeError("core initializer produced an infeasible state")
        population.append(individual)
    return population
```

Population evaluation

```
def evaluate_population (
    population: Sequence[State],
    vehicles: dict[str, Vehicle],
    ship: Ship,
    objective: Callable[[State, dict[str, Vehicle]], float] =
calculate_objective,
) -> tuple[list[float], float, float]:
    if not population:
        raise ValueError("population cannot be empty")

    fitnesses: list[float] = []
    for individual in population:
        if not is_valid_state(individual, vehicles, ship):
            raise ValueError("population contains an infeasible state")
        fitnesses.append(objective(individual, vehicles))

    return fitnesses, max(fitnesses), sum(fitnesses) / len(fitnesses)
```

Parent selection

```
def select_parent_index (
    fitnesses: Sequence[float],
    rng: random.Random,
) -> int:
```

```
    if not fitnesses:
        raise ValueError("fitnesses cannot be empty")
    if any(
        isinstance(fitness, bool)
        or not isinstance(fitness, (int, float))
        or not math.isfinite(fitness)
        or fitness < 0
        for fitness in fitnesses
    ):
        raise ValueError("fitnesses must be finite, nonnegative numbers")

    if not any(fitnesses):
        return rng.randrange(len(fitnesses))

    return rng.choices(range(len(fitnesses)), weights=fitnesses, k=1)[0]
```

Cross-over

```
def crossover_one_point (
    parent_a: State,
    parent_b: State,
    vehicle_ids: Sequence[str],
    rng: random.Random,
) -> State:
    ids = tuple(vehicle_ids)
    if len(set(ids)) != len(ids):
        raise ValueError("vehicle_ids must not contain duplicates")
    required = set(ids)
    if set(parent_a.placements) != required or set(parent_b.placements) !=
required:
        raise ValueError("both parents must contain exactly the vehicle IDs")

    if len(ids) < 2:
        return parent_a.copy()

    cut = rng.randrange(1, len(ids))
    return State(
        placements={
            vehicle_id: (
                parent_a.placements[vehicle_id]
                if index < cut
                else parent_b.placements[vehicle_id]
            )
            for index, vehicle_id in enumerate(ids)
        }
    )
```

State repair

```
def repair_state (
    candidate: State,
    vehicles: dict[str, Vehicle],
    ship: Ship,
    rng: random.Random,
) -> State:
    if set(candidate.placements) != set(vehicles):
        raise ValueError("candidate must contain exactly the vehicle IDs")

    for placement in candidate.placements.values():
        if placement is None:
            continue
        if (
            not isinstance(placement, Placement)
            or type(placement.x) is not int
            or type(placement.y) is not int
            or not isinstance(placement.orientation, Orientation)
        ):
            raise ValueError("candidate contains a malformed placement")

    repaired = State({vehicle_id: None for vehicle_id in vehicles})
    if not is_valid_state(repaired, vehicles, ship):
        raise ValueError("ship or vehicle data cannot form a feasible empty
state")

    occupied: set[tuple[int, int]] = set()
    used_capacity = 0.0

    def free_cells(vehicle: Vehicle, placement: Placement) -> bool:
        if not is_within_bounds(placement, vehicle, ship):
            return False
        width, length = placement.dimension(vehicle)
        return all(
            (x, y) not in occupied
            for y in range(placement.y, placement.y + length)
            for x in range(placement.x, placement.x + width)
        )

    def place(vehicle_id: str, placement: Placement) -> None:
        nonlocal used_capacity
        repaired.placements[vehicle_id] = placement
        used_capacity += vehicles[vehicle_id].weight
        width, length = placement.dimension(vehicles[vehicle_id])
        occupied.update(
            (x, y)
            for y in range(placement.y, placement.y + length)
            for x in range(placement.x, placement.x + width)
        )

    loaded_ids = sorted(
        (vehicle_id for vehicle_id in vehicles if
candidate.placements[vehicle_id] is not None),
        key=lambda vehicle_id: (-vehicles[vehicle_id].shipping_fee, vehicle_id),
```

```
    )
    for vehicle_id in loaded_ids:
        vehicle = vehicles[vehicle_id]
        inherited = candidate.placements[vehicle_id]
        assert inherited is not None

        if used_capacity + vehicle.weight > ship.max_capacity:
            continue

        if free_cells(vehicle, inherited):
            place(vehicle_id, inherited)
            continue

        other_orientation = (
            Orientation.VERTICAL
            if inherited.orientation == Orientation.HORIZONTAL
            else Orientation.HORIZONTAL
        )
        relocated = False
        for orientation in (inherited.orientation, other_orientation):
            probe = Placement(0, 0, orientation)
            width, length = probe.dimension(vehicle)
            columns = ship.width - width + 1
            rows = ship.length - length + 1
            if columns <= 0 or rows <= 0:
                continue

            count = columns * rows
            start = rng.randrange(count)
            for offset in range(count):
                index = (start + offset) % count
                proposed = Placement(index % columns, index // columns,
orientation)
                if proposed != inherited and free_cells(vehicle, proposed):
                    place(vehicle_id, proposed)
                    relocated = True
                    break
            if relocated:
                break

    if not is_valid_state(repaired, vehicles, ship):
        raise RuntimeError("repair generated an invalid state; core invariants
violated")
    return repaired
```

Mutation

```
def mutate_state (
    state: State,
    vehicles: dict[str, Vehicle],
    ship: Ship,
```

```
    rng: random.Random,
    *,
    max_attempts: int = 32,
) -> State:
    if type(max_attempts) is not int or max_attempts < 1:
        raise ValueError("max_attempts must be a positive integer")
    if not is_valid_state(state, vehicles, ship):
        raise ValueError("state must be feasible before mutation")

    vehicle_ids = tuple(sorted(vehicles))
    if not vehicle_ids:
        return state.copy()

    loaded_ids = tuple(
        vehicle_id for vehicle_id in vehicle_ids
        if state.placements[vehicle_id] is not None
    )
    available_operations = ["move"]
    if len(vehicle_ids) > 1:
        available_operations.append("swap")
    if loaded_ids:
        available_operations.append("rotate")

    for _ in range(max_attempts):
        operation = rng.choice(available_operations)
        proposal = state.copy()
        if operation == "swap":
            a, b = rng.sample(vehicle_ids, 2)
            proposal.placements[a], proposal.placements[b] = (
                state.placements[b], state.placements[a]
            )
        elif operation == "rotate":
            vehicle_id = rng.choice(loaded_ids)
            current = state.placements[vehicle_id]
            assert current is not None
            new_orientation = (
                Orientation.VERTICAL
                if current.orientation == Orientation.HORIZONTAL
                else Orientation.HORIZONTAL
            )
            proposal.placements[vehicle_id] = Placement(
                current.x, current.y, new_orientation
            )
        else:
            vehicle_id = rng.choice(vehicle_ids)
            current = state.placements[vehicle_id]
            if current is not None and rng.random() < MOVE_UNLOAD_PROBABILITY:
                proposal.placements[vehicle_id] = None
            else:
                orientation = (
                    rng.choice(tuple(Orientation))
                    if current is None else current.orientation
                )
```

```
                width, length = Placement(0, 0,
orientation).dimension(vehicles[vehicle_id])
                columns = ship.width - width + 1
                rows = ship.length - length + 1
                if columns <= 0 or rows <= 0:
                    continue
                proposal.placements[vehicle_id] = Placement(
                    x=rng.randrange(columns),
                    y=rng.randrange(rows),
                    orientation=orientation,
                )

        if proposal.placements != state.placements and is_valid_state(
            proposal, vehicles, ship
        ):
            return proposal

    return state.copy()
```

Run

```
def run_genetic_algorithm (
    initial_state: State,
    vehicles: dict[str, Vehicle],
    ship: Ship,
    population_size: int = 50,
    max_generations: int = 200,
    mutation_rate: float = 0.1,
    seed: int | None = None,
    objective: Callable[[State, dict[str, Vehicle]], float] =
calculate_objective,
) -> AlgorithmResult:
    if type(max_generations) is not int or max_generations < 0:
        raise ValueError("max_generations must be a nonnegative integer")
    if (
        isinstance(mutation_rate, bool)
        or not isinstance(mutation_rate, (int, float))
        or not math.isfinite(mutation_rate)
        or not 0.0 <= mutation_rate <= 1.0
    ):
        raise ValueError("mutation_rate must be a finite number between 0 and 1")

    rng = random.Random(seed)
    start = time.perf_counter()
    population = initialize_population(
        initial_state,
        vehicles,
        ship,
        population_size,
        seed=rng.getrandbits(64),
    )
```

Run

```
    vehicle_ids = tuple(sorted(vehicles))
    fitnesses, maximum, average = evaluate_population(population, vehicles, ship,
objective=objective)
    history = [maximum]
    average_fitness_history = [average]
    best_index = max(range(population_size), key=fitnesses.__getitem__)
    best_state = population[best_index].copy()
    best_score = fitnesses[best_index]
    best_generation = 0

    for generation in range(1, max_generations + 1):
        next_population: list[State] = []
        for _ in range(population_size):
            parent_a = population[select_parent_index(fitnesses, rng)]
            parent_b = population[select_parent_index(fitnesses, rng)]
            child = crossover_one_point(parent_a, parent_b, vehicle_ids, rng)
            child = repair_state(child, vehicles, ship, rng)
            if rng.random() < mutation_rate:
                child = mutate_state(child, vehicles, ship, rng)
            next_population.append(child)

        population = next_population
        fitnesses, maximum, average = evaluate_population(population, vehicles,
ship, objective=objective)
        history.append(maximum)
        average_fitness_history.append(average)
        generation_best_index = max(range(population_size),
key=fitnesses.__getitem__)
        generation_best_score = fitnesses[generation_best_index]
        if generation_best_score > best_score:
            best_state = population[generation_best_index].copy()
            best_score = generation_best_score
            best_generation = generation

    final_index = max(range(population_size), key=fitnesses.__getitem__)
    final_state = population[final_index].copy()
    return AlgorithmResult(
        algorithm="Genetic Algorithm",
        initial_state=initial_state.copy(),
        final_state=final_state,
        best_state=best_state,
        best_score=best_score,
        runtime=time.perf_counter() - start,
        iterations=max_generations,
        history=history,
        metadata={
            "population_size": population_size,
            "max_generations": max_generations,
            "mutation_rate": mutation_rate,
            "seed": seed,
            "average_fitness_history": average_fitness_history,
```

Run

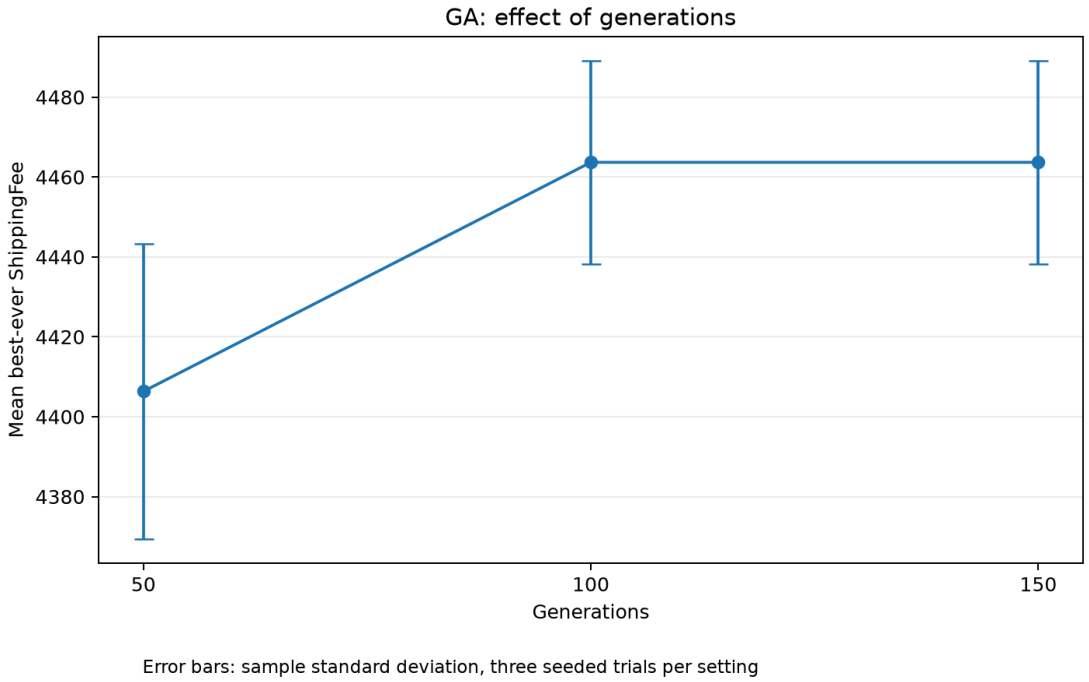
```
            "best_generation": best_generation,
            "final_population_best_score": fitnesses[final_index],
            "fitness_evaluations": population_size * (max_generations + 1),
        },
    )
```

Tabel 2.6.b: Studi Parameter *Genetic Algorithm*

Studi	*Popu-lation*	*Genera-tions*	*Trials*	*Mean Best*	SSD Best	*Mean Final*	*Mean Runtime (s)*
Variasi generasi	40	50	3	4406,33	36,95	4305,00	2,731
Variasi generasi	40	100	3	**4463,67**	**25,40**	4323,00	4,895
Variasi generasi	40	150	3	**4463,67**	**25,40**	4315,00	10,458
Variasi generasi	20	100	3	4237,00	87,18	3911,00	2,006
Variasi generasi	40	100	3	**4463,67**	**25,40**	4323,00	6,696
Variasi generasi	60	100	3	4368,33	27,15	4236,67	7,951

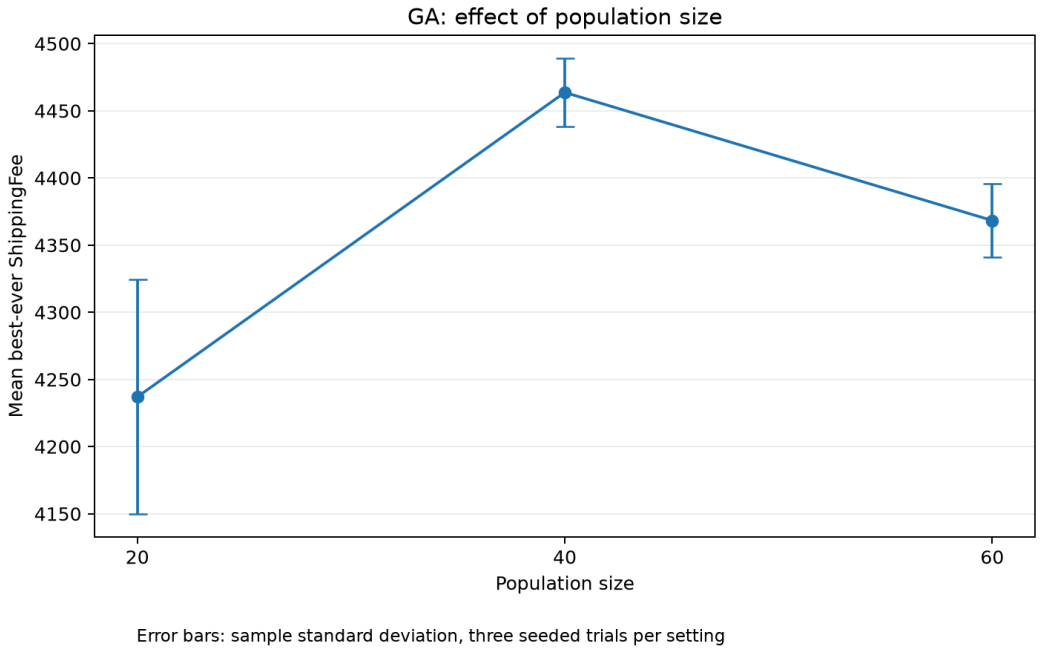
Sumber: output/analysis/synthetic_10×20_pilot/ga_statistics.csv

Konfigurasi 40×100 muncul pada kedua studi. Untuk *seed* yang sama, seluruh *history* maksimum dan rata-rata *fitness* dari kedua pelaksanaan tersebut identik, tetapi dengan *runtime* yang berbeda. Oleh karena itu, dua kemunculan konfigurasi tersebut tidak dianggap enam *trial* independen. Untuk perbandingan utama terhadap *Hill Climbing*/*Simulated Annealing*, *analysis pipeline* memilih tiga *run* 40×100 dari studi variasi generasi.



Gambar 2.6.a. Pengaruh jumlah generasi terhadap *mean best-ever* ShippingFee

Dengan populasi 40, peningkatan 50 menjadi 100 generasi menaikkan rata-rata *best fitness* dari 4.406,33 menjadi 4.463,67, yaitu 57,33 poin atau sekitar 1,30%. Peningkatan menjadi 150 generasi tidak menghasilkan perbaikan *best fitness* pada ketiga *seed*. Pada saat yang sama, *runtime* rata-rata naik dari 4,895 menjadi 10,458 detik, atau sekitar 113,7%. Dengan demikian, untuk *dataset* ini, tambahan 50 generasi setelah generasi ke-100 meningkatkan biaya komputasi tanpa memberi peningkatan kualitas terbaik yang teramati.



Gambar 2.6.b. Pengaruh ukuran populasi terhadap mean best-ever ShippingFee

Dengan 100 generasi, populasi 20 menghasilkan rata-rata 4.237,00. Peningkatan menjadi populasi 40 menaikkan rata-rata sebesar 226,67 poin atau sekitar 5,35%, sekaligus menurunkan *sample standard deviation* dari 87,18 menjadi 25,40. Populasi 60 tidak memberikan peningkatan lanjutan dan rata-rata *best fitness* justru turun menjadi 4.368,33. Hasil tersebut tidak berarti bahwa populasi 40 merupakan optimum universal, tetapi menunjukkan bahwa pada *dataset*, operator, *mutation rate*, dan tiga *seed* yang diuji, populasi 40 memberikan hasil terbaik di antara nilai yang dievaluasi.

Konfigurasi populasi 40 dan 100 generasi muncul dalam kedua studi. Untuk *seed* yang sama, *trajectory fitness*-nya identik, tetapi *runtime* tercatat berbeda karena merupakan pelaksanaan terpisah. Dua kemunculan tersebut karena itu tidak diperlakukan sebagai enam trial independen. Perbandingan utama menggunakan tiga run 40 × 100 dari studi variasi generasi.

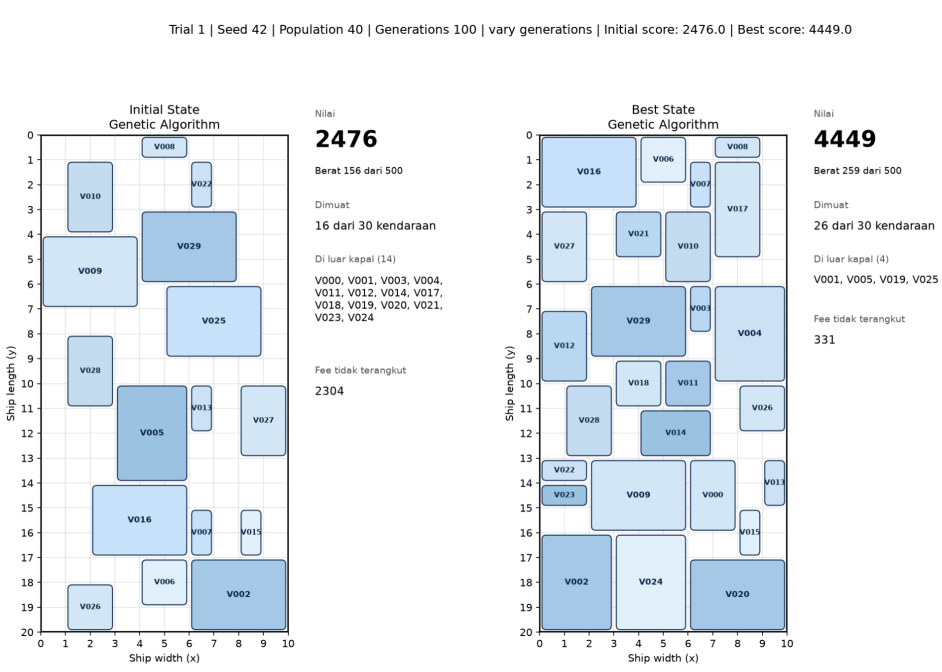
Tabel 2.6.c: *Trial Genetic Algorithm* pada Konfigurasi Terpilih

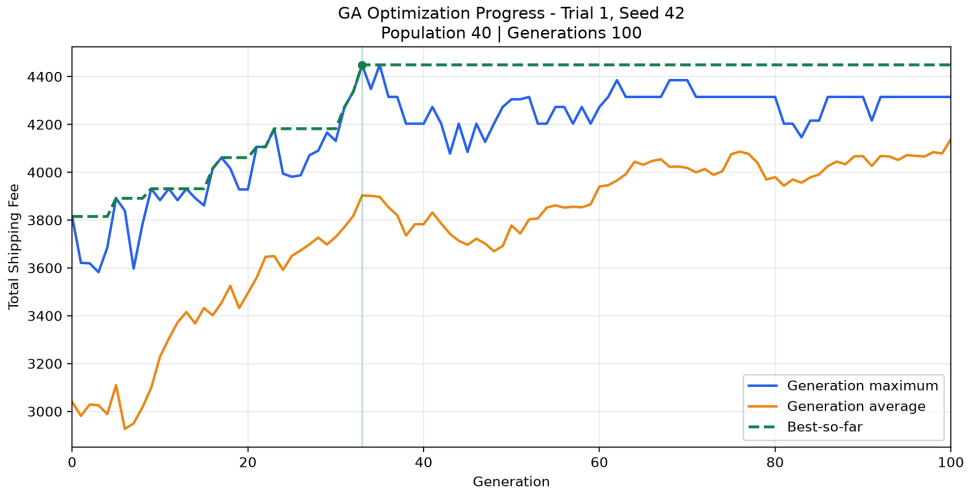
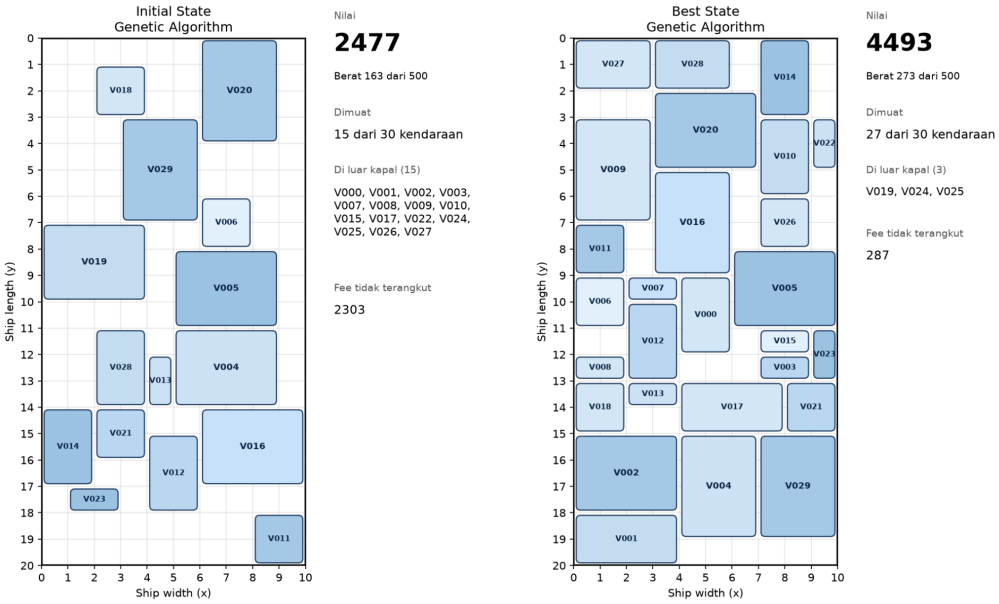
Trial	*Seed*	*Initial*	*Final*	*Best-ever*	*Best Generation*	*Fitness Evaluation*	*Runtime(s)*
1	42	2476	4315	4449	33	4040	4,028
2	43	2477	4339	**4493**	96	4040	4,575
3	44	2933	4315	4449	51	4040	6,081
Mean			**4323,00**	**4463,67**	**60,0**	**4040**	**4,895**

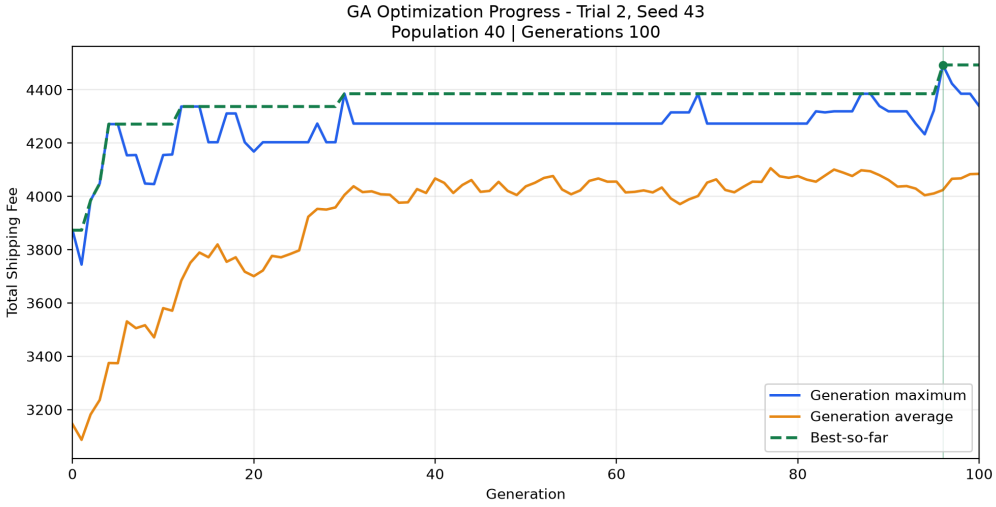
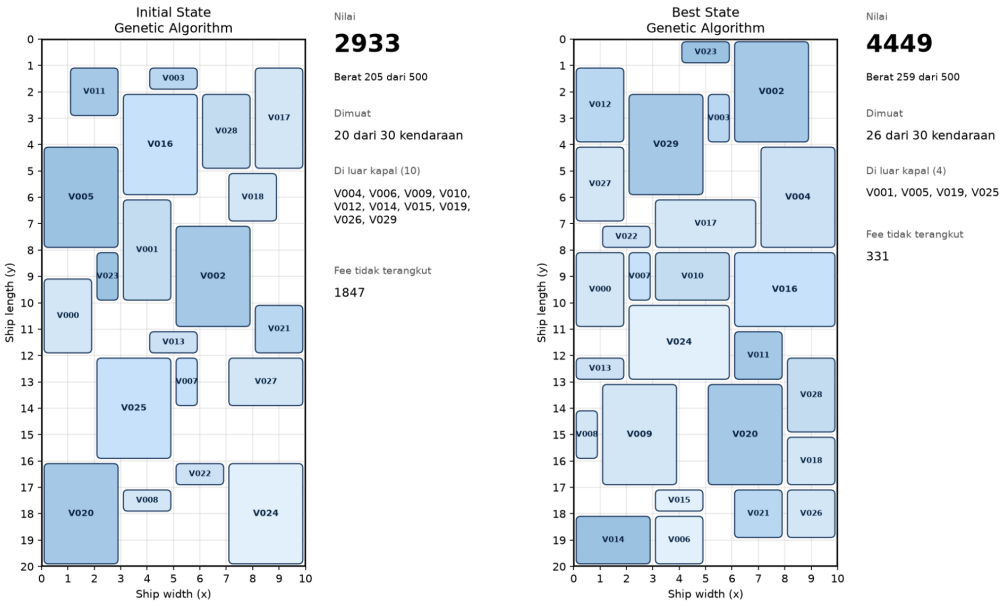
Sumber: output/ga/synthetic_10×20_pilot/ga_summary.csv

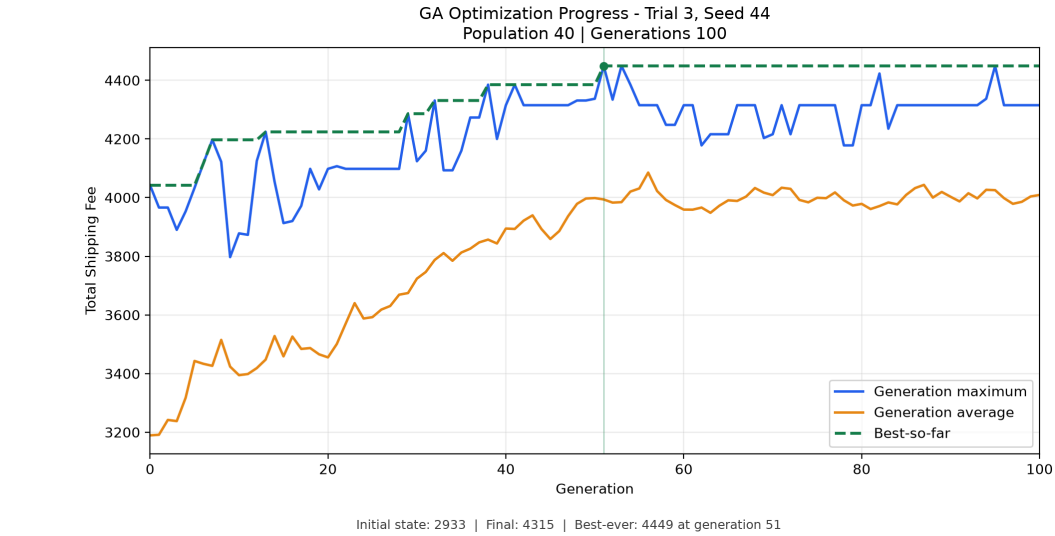
Perbedaan best_score dan final_score pada ketiga *run* berturut-turut adalah 134, 154, dan 134, atau rata-rata 140,67. Fenomena ini terjadi karena tidak adanya *elitism*.

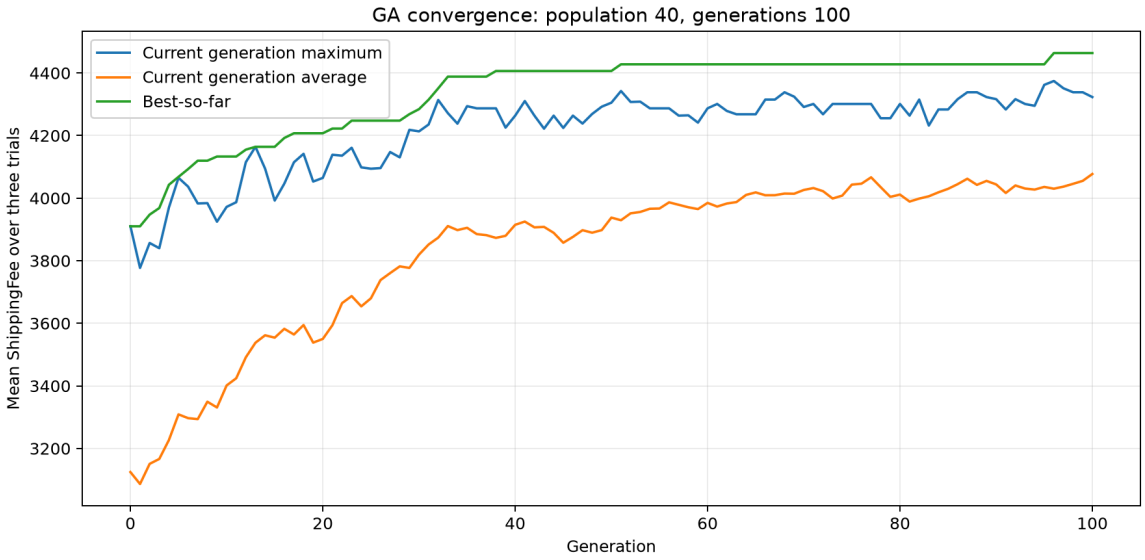
Tabel 2.6.d: Visualisasi *Initial State* dan *Best-Ever State Genetic Algorithm* pada Populasi 40 dan 100 Generasi

Trial	***Initial State* dan *Best State***
1	
	Optimization Progress

Trial	*Initial State* dan *Best State*
	 Initial state: 2476 | Final: 4315 | Best-ever: 4449 at generation 33
2	Trial 2 | Seed 43 | Population 40 | Generations 100 | vary generations | Initial score: 2477.0 | Best score: 4493.0 
	Optimization Progress

Trial	*Initial State* dan *Best State*
	 Initial state: 2477 | Final: 4339 | Best-ever: 4493 at generation 96
3	Trial 3 | Seed 44 | Population 40 | Generations 100 | vary generations | Initial score: 2933.0 | Best score: 4449.0 
	Optimization Progress

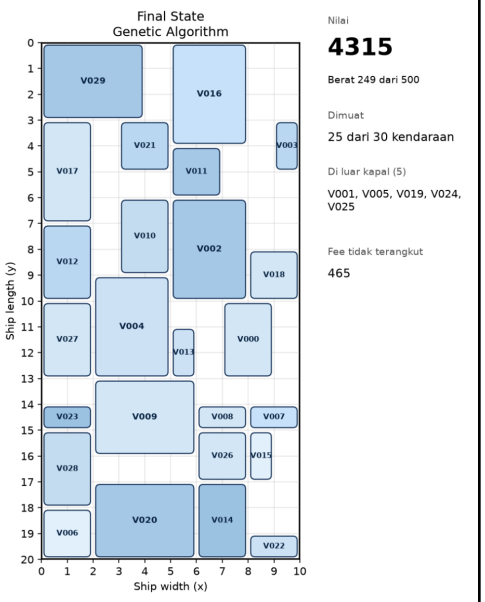
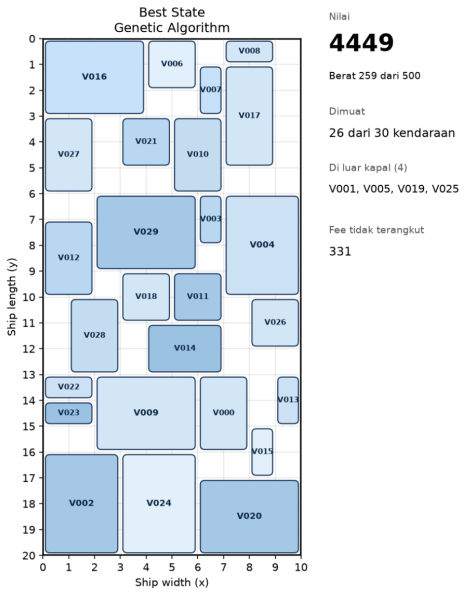
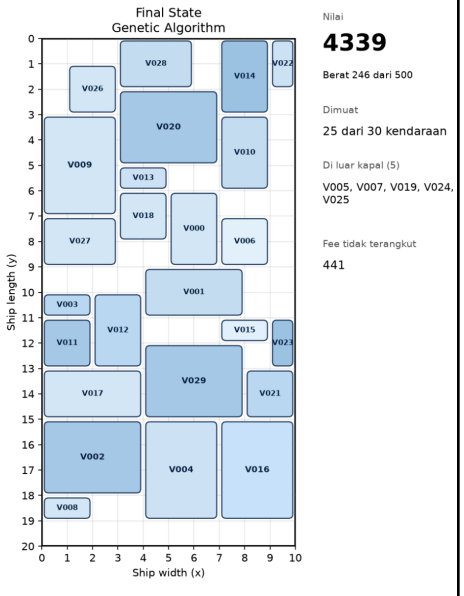
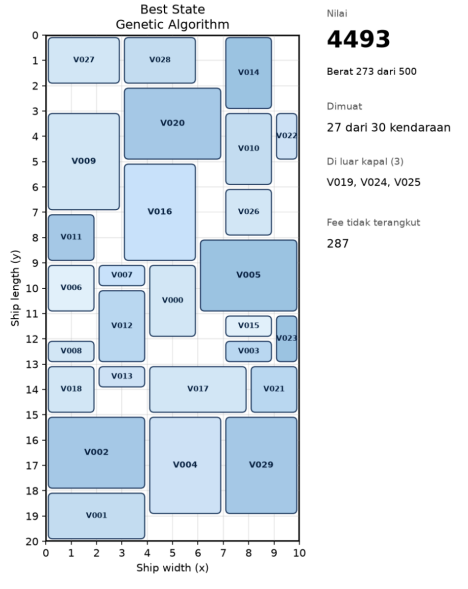
Trial	*Initial State* dan *Best State*
	

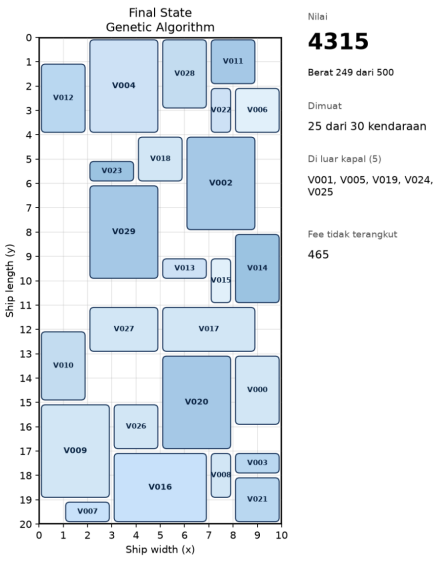
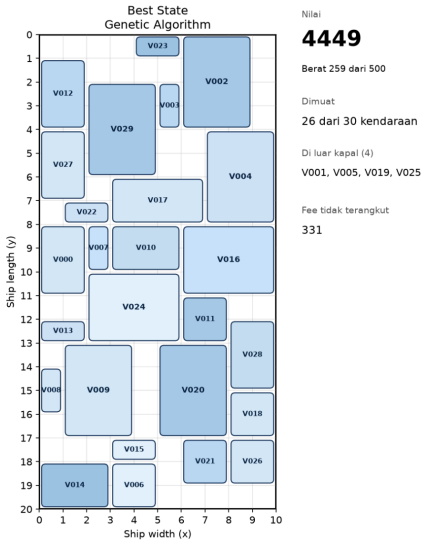


Gambar 2.6.c. Konvergensi *Genetic Algorithm* pada *population size* 40 dan 100 *generations*

Kurva *current-generation maximum* tidak harus monoton karena implementasi tidak menggunakan elitism. Sebaliknya, *best-so-far* monoton tidak menurun karena merupakan pencatatan solusi terbaik sepanjang run. Kurva rata-rata populasi meningkat secara umum, yang menunjukkan tekanan seleksi mengarahkan populasi menuju konfigurasi dengan *fitness* lebih tinggi.

Tabel 2.6.e: Perbandingan *Final State* dan *Best-ever State Genetic Algorithm* pada Populasi 40 dan 100 Generasi

Trial/Seed	***Final State* – Generasi 100**	***Best-ever State* – Generasi 33**
1/42	vary generations | Initial score: 2476.0 | Final score: 4315.0 	vary generations | Initial score: 2476.0 | Best score: 4449.0 
	Fitness: **4315**	Fitness: **4449**
2/43	00 | vary generations | Initial score: 2477.0 | Final score: 4339.0 	vary generations | Initial score: 2477.0 | Best score: 4493.0 
	Fitness: **4339**	Fitness: **4493**

Trial/Seed	*Final State* – Generasi 100	*Best-ever State* – Generasi 33
3/44	| vary generations | Initial score: 2933.0 | Final score: 4315.0 	| vary generations | Initial score: 2933.0 | Best score: 4449.0 
	Fitness: **4315**	Fitness: **4449**

Pada trial 1 dengan seed 42, Genetic Algorithm menemukan best-ever state dengan fitness 4449 pada generasi ke-33. Sementara itu, individu terbaik pada generasi terakhir memiliki fitness 4315. Selisih sebesar 134 poin menunjukkan bahwa solusi terbaik yang pernah ditemukan tidak selalu bertahan dalam populasi terakhir. Kondisi ini sesuai dengan mekanisme pure generational replacement tanpa elitism. Oleh karena itu, best_state disimpan secara terpisah sebagai solusi terbaik sepanjang pencarian, sedangkan final_state mendokumentasikan hasil populasi terakhir.

Perbedaan antara *best score* dan *final score* untuk tiga *trial* Genetic Algorithm adalah 134, 154, dan 134, atau rata-rata 140,67 poin. Hal ini bukan kesalahan pencatatan, melainkan konsekuensi *pure generational replacement*, di mana solusi terbaik dapat hilang dari populasi berikutnya, sedangkan best_state global tetap disimpan. Untuk rekomendasi kepada pengguna, best_state lebih tepat digunakan karena merupakan solusi dengan *objective* tertinggi yang benar-benar pernah ditemukan algoritma. final_state tetap perlu dicatat untuk memenuhi transparansi eksperimen.

2.7. ETA-Weighted Objective

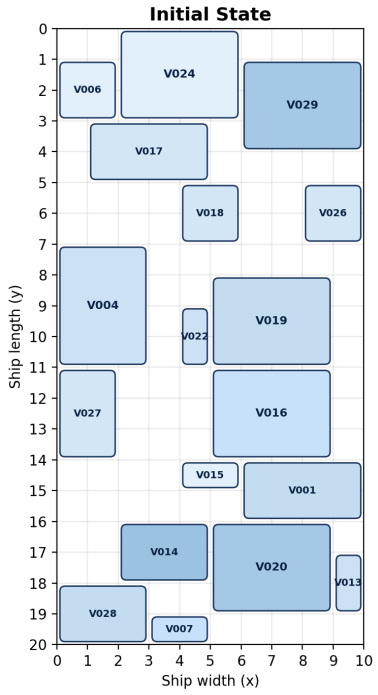
Tabel 2.7.a: *Perbandingan hasil optimasi dengan ETA-Weighted Objective*

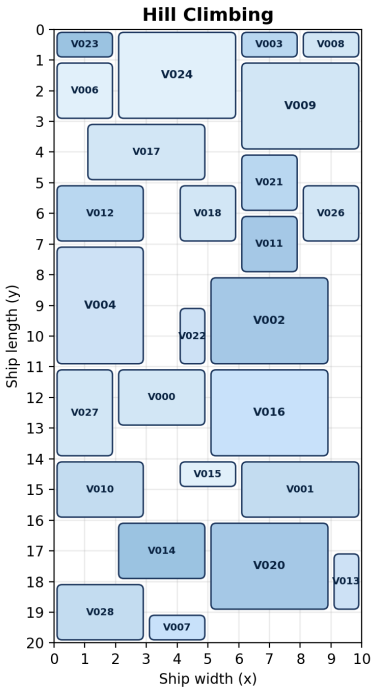
Algorithm	Initial Objective	Best Objective	Improvement (%)	Actual Shipping Fee	Runtime (s)	Iterations
Hill Climbing	3369.8	5404.7	60.39	4348	2.4061	11
Simulated Annealing		5210.4	54.62	4214	9.073	1000
Genetic Algorithm		5575.7	65.46	4519	16.899	1000
Hill Climbing (Sideways)		5404.7	60.39	4348	7.1846	31
Hill Climbing (Stochastic)		5327.7	58.1	4278	5.1593	23
Hill Climbing (Random Restart)		5404.7	60.39	4348	9.3589	41

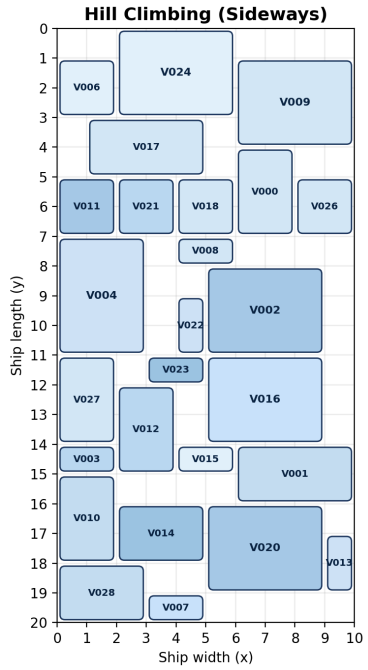
Penggunaan ETA-Weighted Objective menghasilkan nilai optimasi yang berbeda pada setiap algoritma. Hal ini disebabkan oleh penambahan faktor urgensi berdasarkan ETA kendaraan, sehingga pemilihan kendaraan tidak hanya mempertimbangkan besarnya ShippingFee, tetapi juga memberikan prioritas lebih tinggi kepada kendaraan dengan ETA lebih kecil. Dari seluruh algoritma yang diuji, Genetic Algorithm menghasilkan best objective tertinggi sebesar 5575,7 dengan peningkatan 65,46% dari initial objective. Algoritma ini juga menghasilkan actual ShippingFee tertinggi sebesar 4519, meskipun membutuhkan waktu komputasi paling lama, yaitu 16,899 detik. Sementara itu, Steepest-Ascent Hill-Climbing mencapai nilai objektif sebesar 5404,7 hanya dalam 11 iterasi dengan waktu 2,4061 detik, menunjukkan efisiensi pencarian yang lebih tinggi. Perbedaan hasil tersebut menunjukkan bahwa setiap algoritma memiliki karakteristik pencarian yang berbeda dalam mengoptimalkan kombinasi nilai pengiriman dan urgensi kendaraan. Selain itu, nilai best objective pada pendekatan ETA-Weighted tidak dapat langsung dibandingkan sebagai pendapatan

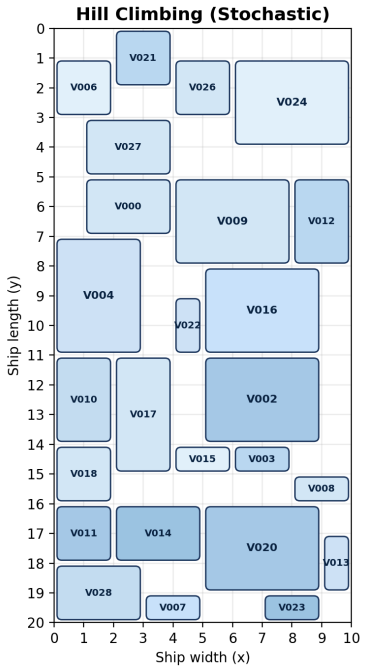
aktual karena telah mencakup bobot urgensi, sehingga actual ShippingFee tetap digunakan sebagai indikator nilai ekonomi pengiriman.

Tabel 2.7.b: *Visualisasi hasil optimasi dengan ETA-Weighted Objective*

Initial State		
		
Steepest Hill-Climbing	**Sideways Move Hill-Climbing**	**Stochastic Hill-Climbing**



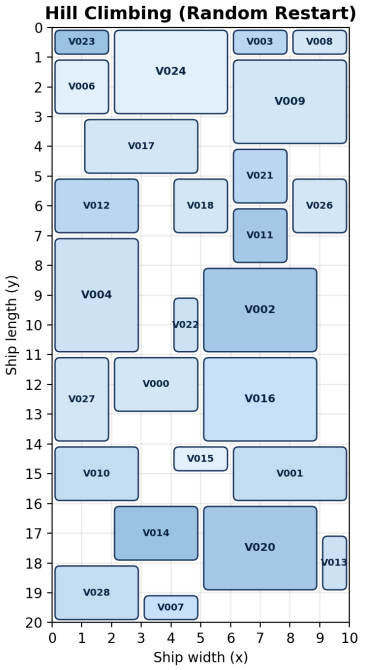


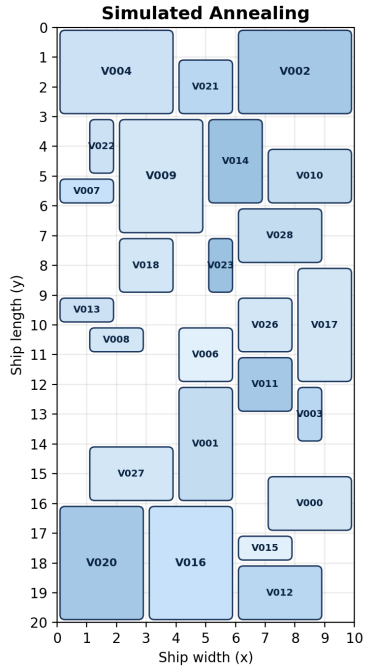


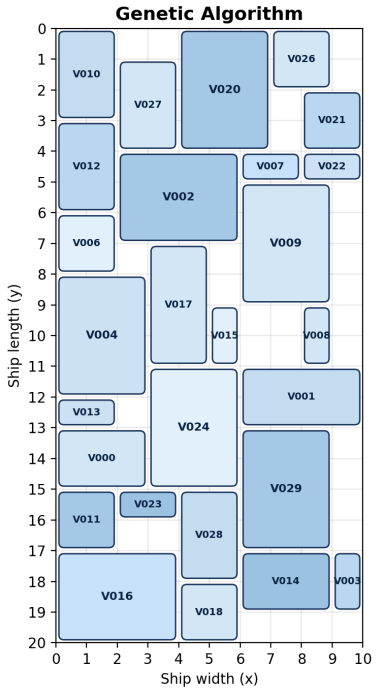
Random Restart Hill-Climbing

Simulated Annealing

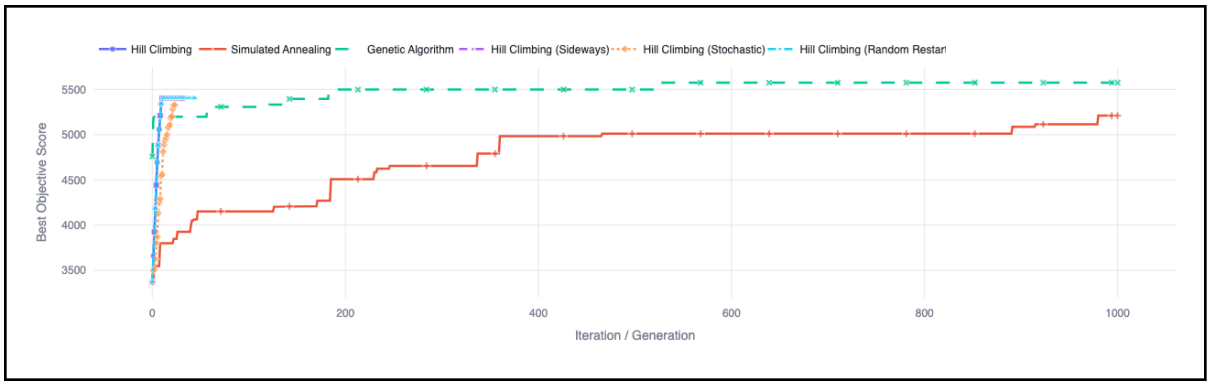
Genetic Algorithm







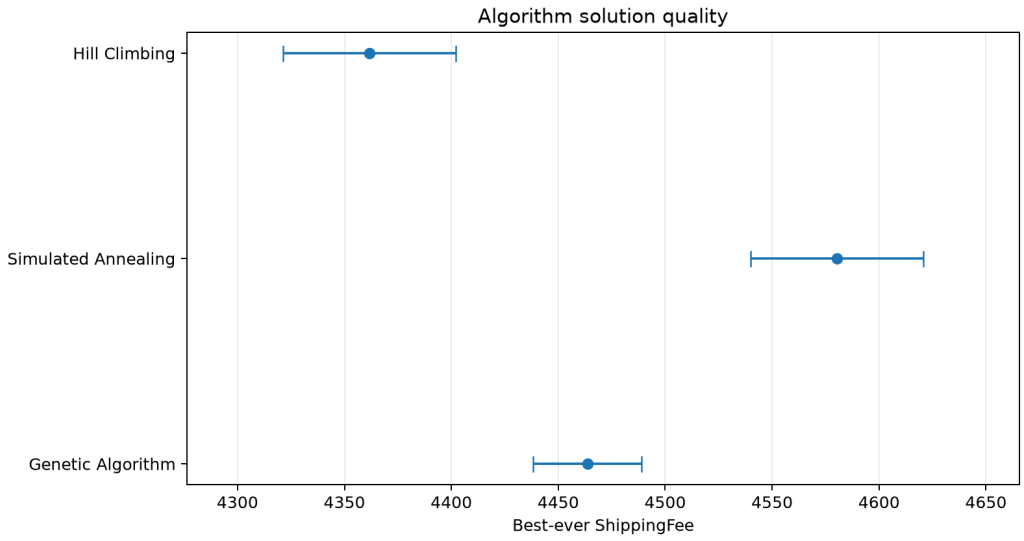
Convergence Comparison



Dengan ETA-Weighted Objective, kendaraan dengan nilai ShippingFee yang tinggi belum tentu selalu diprioritaskan karena tingkat urgensi berdasarkan ETA turut memengaruhi nilai objektif. Akibatnya, algoritma dapat memilih kombinasi kendaraan yang berbeda dibandingkan optimasi yang hanya mempertimbangkan ShippingFee. Meskipun demikian, seluruh konfigurasi tetap harus memenuhi batasan kapasitas berat, dimensi geladak, dan larangan tumpang tindih antarkendaraan. Visualisasi ini menunjukkan bahwa perubahan fungsi objektif dapat memengaruhi keputusan pemuatan dan susunan kendaraan pada kapal, sedangkan perbedaan mekanisme pencarian antarialgoritma turut menentukan konfigurasi akhir yang diperoleh.

2.8. Analisis Eksperimen

Berikut ini perbandingan hasil ketiga algoritma yang menjadi spesifikasi wajib pada tugas besar ini.



Gambar 2.8.a. *Mean best-ever* ShippingFee *Hill Climbing, Simulated Annealing*, dan *Genetic Algorithm* beserta *sample standard deviation*

Tabel 2.8.a: Perbandingan Hasil Ketiga Algoritma

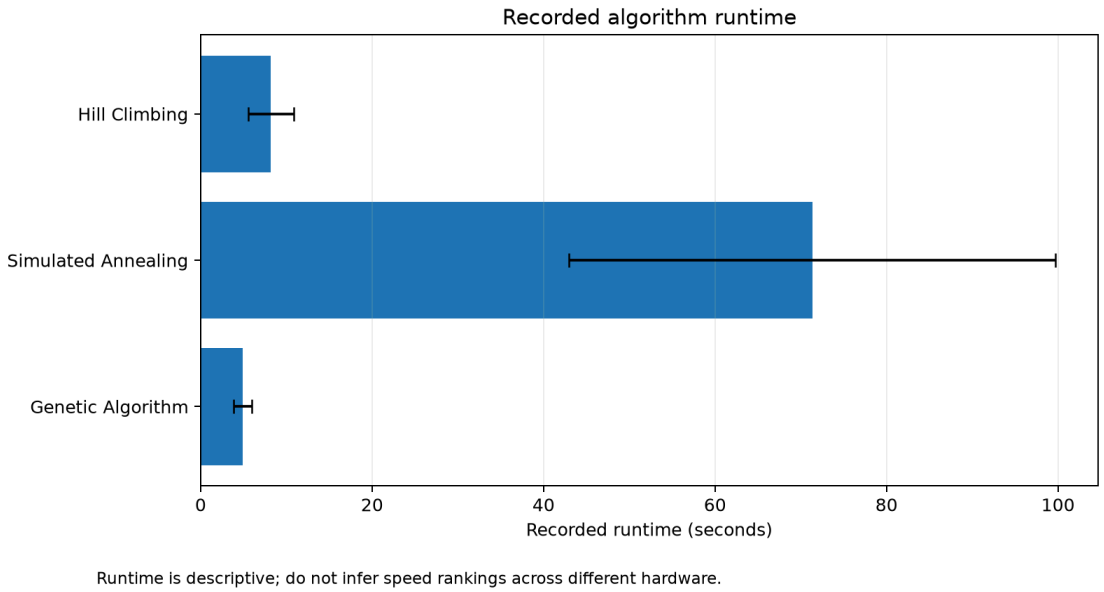
Algoritma	*Trials*	*Mean Best*	*SD Best*	CV	*Mean Final*	*Mean Runtime*	*Mean* Iterasi
Hill Climbing	3	4361,67	40,41	0,927%	4361,67	8,20s	12,0 iterasi
Simulated Annealing	3	**4580,33**	40,41	0,882%	**4580,33**	71,33s	18.417 iterasi
Genetic Algorithm	3	4463,67	**25,40**	**0,569%**	4323,00	**4,89s**	100 generasi

Simulated Annealing memperoleh *best score* tertinggi pada ketiga *seed*. Secara rata-rata, *Simulated Annealing* lebih tinggi 218,67 unit daripada *Hill Climbing*, atau sekitar 5,01% relatif terhadap rerata *Hill Climbing*. *Genetic Algorithm* lebih tinggi 102,00 unit daripada *Hill Climbing*, atau sekitar 2,34%. Selisih rerata *Simulated Annealing* dan *Genetic Algorithm* adalah 116,67 unit atau sekitar 2,61% relatif terhadap *Genetic Algorithm* .

Simpangan baku *Genetic Algorithm* sebesar 25,40 merupakan yang terendah di antara ketiga algoritma pada eksperimen ini. Hal tersebut menunjukkan bahwa hasil *Genetic Algorithm* pada tiga *seed* lebih rapat daripada hasil *Hill Climbing* dan *Simulated Annealing*. Akan tetapi, karena hanya terdapat tiga trial, statistik ini sebaiknya dipahami sebagai ukuran deskriptif, bukan bukti inferensial bahwa *Genetic Algorithm* secara umum lebih stabil.

Runtime rata-rata yang tercatat menempatkan *Genetic Algorithm* pada 4,89 detik, *Hill Climbing* pada 8,20 detik, dan *Simulated Annealing* pada 71,33 detik. Secara numerik, *Simulated Annealing* membutuhkan sekitar 14,6 kali runtime *Genetic Algorithm* pada hasil yang tersedia. Namun, rasio tersebut tidak boleh langsung ditafsirkan sebagai kompleksitas intrinsik algoritma karena anggaran pencariannya berbeda, yaitu *Hill Climbing* pada eksperimen berhenti setelah 31 iterasi pada setiap seed, *Simulated Annealing* menjalankan sekitar 18.416 iterasi sebagai konsekuensi jadwal temperatur, sedangkan *Genetic Algorithm* mengevaluasi populasi sepanjang 100 generasi.

Selain itu, konfigurasi *Genetic Algorithm* populasi 40 dan 100 generasi yang dieksekusi pada dua studi menghasilkan sejarah fitness yang sama untuk setiap *seed*, tetapi rerata *runtime* masing-masing sekitar 4,89 dan 6,70 detik. Variasi ini memperlihatkan bahwa kondisi eksekusi sistem juga memengaruhi pengukuran waktu. Oleh sebab itu, runtime digunakan sebagai data praktis tambahan, bukan dasar tunggal untuk menyatakan algoritma tertentu secara universal lebih efisien.



Gambar 2.8.b. *Runtime* tercatat untuk Hill Climbing, Simulated Annealing, dan *Genetic Algorithm*

Hill Climbing mencapai hasil akhir setelah 31 iterasi untuk ketiga seed. Karakteristik ini konsisten dengan algoritma yang berhenti segera setelah tidak tersedia perbaikan yang diterima. *Simulated Annealing* menggunakan eksplorasi jauh lebih panjang dan tetap dapat menerima perpindahan yang menurunkan objektif selama temperatur memadai. Akibatnya, final_score *Simulated Annealing* tidak selalu sama dengan best_score. Sebagai contoh, pada seed 42 *Simulated Annealing* memiliki final_score 4.138 tetapi pernah memperoleh best_score 4.627. Perbedaan tersebut kembali menegaskan bahwa evaluasi algoritma stokastik harus menggunakan solusi terbaik yang disimpan, bukan hanya keadaan aktif pada iterasi terakhir.

Simulated Annealing mencatat 35, 35, dan 39 episode *stuck* pada *seed* 42, 43, dan 44 menurut metadata eksperimen. Karena metrik tersebut berasal dari pemeriksaan berkala implementasi, nilainya digunakan sebagai diagnostik internal dan bukan jumlah *local optimum* matematis yang berbeda.

Dengan mengabaikan detail bentuk persegi panjang dan hanya mempertahankan batas luas total 200 sel, optimasi subset memberikan batas atas ShippingFee sebesar 4.729. Relaksasi ini disebut batas atas karena solusi yang hanya memenuhi jumlah luas belum tentu dapat ditata secara geometris tanpa tumpang

tindih. Solusi terbaik *Genetic Algorithm* sebesar 4.493 mencapai sekitar 95,01% dari batas atas relaksasi tersebut, sedangkan solusi *Simulated Annealing* terbaik 4.627 lebih dekat lagi. Karena batas atas mengabaikan kendala packing geometris, angka 4.729 bukan nilai optimum global yang telah dibuktikan untuk persoalan lengkap.

2.8.1. Kualitas Solusi

Berdasarkan *best solution encountered*, Simulated Annealing menempati posisi pertama dengan mean 4580,33, Genetic Algorithm kedua dengan 4463,67, dan Hill Climbing ketiga dengan 4361,67. Dibanding Hill Climbing, Genetic Algorithm meningkatkan mean best score sebesar 102 atau sekitar 2,34%. Simulated Annealing meningkatkan mean best sebesar 116,67 atau sekitar 2,61% terhadap Genetic Algorithm, dan 218,67 atau sekitar 5,01% terhadap Hill Climbing.

Karena *initial state* dipasangkan berdasarkan *seed*, perbedaan dapat diperiksa per *trial*.

Tabel 2.8.1.a: Perbandingan Kualitas Solusi

Perbandingan	Seed 42	Seed 43	Seed 44	Mean Delta
GA - HC	+134	+108	+64	**+102,00**
SA -GA	+108	+134	+108	**+116,67**
SA - HC	+242	+242	+172	**+218,67**

Semua *paired difference* pada tiga *seed* memiliki arah yang sama, yaitu GA > HC dan SA > GA untuk metrik *best-ever*. Akan tetapi, dengan hanya tiga pasangan, laporan ini tidak menjadikan uji signifikansi parametrik sebagai dasar kesimpulan. *Mean*, *sample* SD, dan *paired* delta lebih tepat diperlakukan sebagai bukti deskriptif.

2.8.2. *Final State versus Best State*

Perbedaan penting muncul ketika yang dibandingkan adalah final state sesuai terminologi spesifikasi, bukan solusi terbaik sepanjang proses. *Hill climbing* dan *Simulated Annealing* pada *output* ini memiliki final_score =

best_score. Di sisi lain, *Genetic Algorithm* menghasilkan $\bar{Best}_{GA} = 4463,67$ dan $\bar{Final}_{GA} = 4463,67$.

Dengan demikian, apabila penilaian murni menggunakan individu terbaik di populasi terakhir, *mean* final *Genetic Algorithm* justru 38,67 poin lebih rendah daripada *Hill Climbing*. Sebaliknya, jika sistem memberikan solusi terbaik yang pernah ditemukan kepada pengguna, *Genetic Algorithm* mengungguli *Hill Climbing* sebesar 102 poin.

Hal ini bukan inkonsistensi data, melainkan konsekuensi desain *replacement Genetic Algorithm*. *Source* final mengganti seluruh populasi setiap generasi tanpa menyisipkan *elite* lama, tetapi menyimpan best_state global secara terpisah.

Interpretasi ini sebaiknya ditulis eksplisit dalam laporan dan dijelaskan saat demo. Untuk aplikasi keputusan, best_state adalah state yang paling masuk akal ditawarkan sebagai rekomendasi karena memiliki objective tertinggi yang telah ditemukan. Untuk memenuhi dokumentasi eksperimen, final_state tetap harus dilaporkan.

2.8.3. Kedekatan terhadap *Global Optimum*

Total ShippingFee semua kendaraan adalah 4.780. Nilai tersebut dapat dipakai sebagai *loose upper bound* karena solusi valid tidak mungkin mempunyai ShippingFee lebih tinggi daripada jumlah seluruh kendaraan. Akan tetapi, luas total kendaraan adalah 206 sel, sedangkan luas kapal hanya 200 sel. Dengan demikian, 4.780 bukan optimum *feasible* dan tidak boleh disebut "*global optimum*".

$$f(s) \leq f^{*} \leq U,\ U = \sum_{i \in V} f_{i} = 4780$$

Rata-rata *best score Hlll Climbing, Genetic Algorithm,* dan *Simulated Annealing* berturut-turut setara 91,25%, 93,38%, dan 95,82% dari *loose upper bound*. Solusi individual *Simulated Annealing* sebesar 4.627 setara 96,80% dari batas atas. Persentase tersebut merupakan rasio terhadap batas atas, bukan

bukti bahwa persentase optimum global yang tersisa telah diketahui secara tepat.

Jika sebuah solusi *feasible* bernilai v dan batas atas U sah, selisih $U - v$ membatasi *gap* absolut optimum dari atas. Untuk v = 4.627 dan U = 4.780, *gap* absolut paling besar 153 dalam satuan *objective*. Besarnya *gap* sebenarnya dapat lebih kecil. Selisih relatif terhadap U sebesar 3,20% juga bukan pengukuran langsung *gap* terhadap optimum yang belum diketahui.

Batas atas yang lebih ketat dapat diperoleh melalui relaksasi *subset* yang hanya mempertahankan luas dan berat. Dalam relaksasi tersebut, pilih z_i biner untuk memaksimalkan jumlah $f_i z_i$ dengan jumlah luas tidak melebihi WL dan jumlah berat tidak melebihi C. Semua *layout feasible* memenuhi relaksasi, sedangkan solusi relaksasi belum tentu dapat ditata secara geometris. Oleh sebab itu, optimum relaksasi sah sebagai batas atas masalah lengkap.

$$Uarea = max\left\{\sum_{i\in V} f_i z_i : \sum_{i\ni V} w_i l_i z_i \leq WL,\ \sum_{i\in V} b_i z_i \leq C\right\}$$

Nilai optimum relaksasi perlu disertai dataset, formulasi, metode penyelesaian, dan hasil *solver*. Pembuktian optimum global masalah geometris memerlukan *solver* eksak dengan status optimal atau bukti batas yang berimpit dengan solusi *feasible*. Kenaikan kurva yang berhenti atau hasil tiga *run* yang sama tidak cukup sebagai bukti optimum global.

2.8.4. Konsistensi

Genetic Algorithm mempunyai *sample standard deviation* terkecil, yaitu 25,40, dan CV 0,569%. Simulated Annealing dan Hill Climbing sama-sama mempunyai SD 40,41, dengan CV masing-masing 0,882% dan 0,927%. Berdasarkan tiga trial yang tersedia, Genetic Algorithm karena itu merupakan algoritma paling konsisten untuk metrik best-ever ShippingFee. Akan tetapi, ukuran sampel yang kecil tetap membatasi generalisasi.

2.8.5. Pengaruh Jumlah Generasi *Genetic Algorithm*

Dari 50 menuju 100 generasi, mean best meningkat dari 4406,33 menjadi 4463,67. Peningkatan tersebut menunjukkan bahwa 50 generasi belum sepenuhnya memanfaatkan proses evolusi untuk tiga seed. Namun, 150 generasi tidak meningkatkan best score sama sekali dibanding 100 generasi. Bahkan best generation dari trial 40 × 100 berada pada generasi 33, 96, dan 51. Setelah solusi tersebut ditemukan, generasi tambahan tidak menemukan improvement lebih tinggi. Dengan demikian, konfigurasi 150 generasi menghasilkan diminishing returns pada dataset ini. Runtime mean studi generasi meningkat dari 4,895 detik menjadi 10,458 detik tanpa peningkatan mean best.

2.8.6. Pengaruh Ukuran Populasi *Genetic Algorithm*

Populasi 20 memiliki hasil paling tidak konsisten dengan SD 87,18 dan mean best 4237,00. Populasi 40 meningkatkan mean best menjadi 4463,67 dan menurunkan SD menjadi 25,40. Secara intuitif terhadap implementasi proyek, populasi lebih besar memberikan lebih banyak kandidat untuk selection dan crossover setiap generasi. Namun hasil pada populasi 60 menunjukkan bahwa "lebih besar" tidak otomatis berarti "lebih baik": mean best turun menjadi 4368,33. Salah satu penjelasan yang konsisten dengan desain eksperimen adalah bahwa budget generasi tetap 100 dan operator stokastik mengikuti trajectory yang berbeda ketika ukuran populasi berubah. Selain itu, lebih banyak individu meningkatkan jumlah fitness evaluation menjadi:

- P=20, G=100: 2020 evaluasi,
- P=40, G=100: 4040 evaluasi, dan
- P=60, G=100: 6060 evaluasi.

Pada eksperimen ini, tambahan evaluasi dari 4040 ke 6060 tidak menghasilkan solusi lebih baik.

2.8.7. Mengapa *Simulated Annealing* Mendapatkan Solusi Terbaik

Hasil eksperimen konsisten dengan mekanisme Simulated Annealing: Hill Climbing berhenti ketika tidak ada neighbor yang lebih baik, sedangkan Simulated Annealing sesekali dapat menerima kandidat yang lebih buruk ketika

temperatur masih tinggi. Mekanisme tersebut memberi Simulated Annealing peluang untuk melewati lembah lokal dan menjelajahi wilayah state space yang tidak dapat dicapai oleh Hill Climbing monoton. Prinsip penerimaan solusi buruk secara terkontrol merupakan alasan utama Simulated Annealing dikembangkan untuk optimasi kombinatorial. Pada eksperimen ini, kemampuan eksplorasi tersebut datang dengan biaya 18.417 iterasi dan runtime yang jauh lebih besar.

Namun, operator Genetic Algorithm yang dipakai harus melalui proses repair agar memenuhi constraints geometri. Repair dapat menghilangkan atau memindahkan gen hasil crossover, sehingga pewarisan komponen baik dari parent tidak selalu dipertahankan utuh. Selain itu, ketiadaan elitism menyebabkan individu terbaik dapat hilang dari populasi aktif. Kedua faktor tersebut merupakan penjelasan yang masuk akal untuk mengapa Genetic Algorithm pada konfigurasi ini belum menyamai Simulated Annealing meskipun tetap mengungguli Hill Climbing. Pernyataan ini merupakan interpretasi terhadap desain algoritma dan hasil eksperimen, bukan bukti kausal formal.

2.8.8. Ancaman terhadap Validitas

Terdapat beberapa batasan penting. Pertama, setiap konfigurasi hanya mempunyai tiga trial, sehingga estimasi variabilitas masih kasar. Kedua, evaluasi difokuskan pada satu dataset sintetik, sehingga perilaku algoritma pada ukuran kapal, jumlah kendaraan, distribusi dimensi, dan rasio kapasitas berbeda belum diuji. Ketiga, anggaran komputasi ketiga algoritma tidak disetarakan berdasarkan jumlah evaluasi objective sehingga runtime bukan perbandingan kompleksitas yang terkontrol. Keempat, optimum global eksak belum diketahui. Kelima, studi Genetic Algorithm hanya mengubah populasi dan generasi, sedangkan mutation rate dipertahankan 0,1. Oleh karena itu, hasil tidak membuktikan bahwa 0,1 adalah nilai mutation optimal. Keenam, ketiadaan elitism pada Genetic Algorithm membuat final-state fitness tidak sama dengan best-ever fitness. Seluruh batasan tersebut perlu dinyatakan secara eksplisit agar interpretasi hasil tidak melampaui bukti yang tersedia.

BAB 3
KESIMPULAN DAN SARAN

Berdasarkan implementasi dan eksperimen yang telah dilakukan, ketiga algoritma berhasil menghasilkan state feasible dengan nilai objective yang lebih tinggi dibandingkan initial state pada dataset synthetic_10×20. Hal ini menunjukkan bahwa representasi state, objective function, constraints, dan operasi neighbor yang dibangun dapat digunakan bersama oleh beberapa strategi pencarian. Spesifikasi wajib satu varian Hill Climbing, Simulated Annealing, dan Genetic Algorithm juga telah tercakup dalam eksperimen.

Dalam hal kualitas solusi terbaik, Simulated Annealing merupakan metode terbaik pada eksperimen dengan rata-rata best-ever ShippingFee 4.580,33. Genetic Algorithm berada pada posisi kedua dengan 4.463,67 dan Steepest-Ascent Hill Climbing pada posisi ketiga dengan 4.361,67. Selisih Genetic Algorithm terhadap Hill Climbing adalah +102 poin secara rata-rata, sedangkan Simulated Annealing mengungguli Genetic Algorithm sebesar +116,67 poin. Ketiga seed menghasilkan arah perbedaan yang sama.

Dalam hal konsistensi, Genetic Algorithm memberikan variasi terkecil dengan sample standard deviation 25,40 dan CV 0,569%. Dalam hal runtime tercatat, Genetic Algorithm 40 × 100 mempunyai nilai rata-rata terendah sebesar 4,895 detik, dibandingkan 8,199 detik pada Hill Climbing dan 71,330 detik pada Simulated Annealing. Namun, runtime tersebut hanya menjadi deskripsi pelaksanaan eksperimen dan tidak cukup untuk menyimpulkan kompleksitas komputasi relatif karena satu iterasi masing-masing algoritma mempunyai pekerjaan yang berbeda.

Studi parameter Genetic Algorithm menunjukkan bahwa konfigurasi population size = 40, generations = 100, mutation rate = 0,1 merupakan pilihan paling masuk akal dari konfigurasi yang diuji. Dibandingkan 50 generasi, 100 generasi meningkatkan rata-rata best fitness. Sebaliknya, 150 generasi tidak memberikan peningkatan lagi dan hanya meningkatkan runtime. Populasi 40 juga memberikan kualitas lebih tinggi daripada populasi 20 dan 60. Konfigurasi tersebut karena itu direkomendasikan sebagai konfigurasi Genetic Algorithm final untuk dataset ini, tetapi tidak diklaim sebagai hyperparameter optimum universal.

Perbedaan antara final fitness dan best-ever fitness Genetic Algorithm memperlihatkan konsekuensi penting dari pure generational replacement. Pada tiga trial terpilih, best state masing-masing lebih baik 134, 154, dan 134 poin daripada final state. Sistem sudah mengatasi masalah dari sisi keluaran dengan menyimpan best_state secara terpisah. Untuk pengembangan lebih lanjut, elitism layak diuji agar sebagian individu terbaik juga dipertahankan dalam populasi, kemudian dibandingkan secara empiris terhadap desain sekarang.

Bonus varian Hill Climbing menunjukkan bahwa Random-Restart memberikan kualitas tertinggi di antara varian Hill Climbing yang diuji, sedangkan objective ETA-weighted berhasil memperluas definisi kualitas solusi dengan mempertimbangkan urgensi tanpa menambah atribut baru. Kedua implementasi tersebut sesuai dengan dua spesifikasi bonus pertama.

Prioritas penyelesaian adalah menyamakan source, manifest, tabel, dan visualisasi pada satu versi eksperimen. Sesudah bukti konsisten, perluasan evaluasi dapat dilakukan dengan lebih banyak seed dan dataset yang membedakan keterbatasan ruang, berat, dan bentuk. Untuk membandingkan strategi pencarian secara lebih adil, gunakan anggaran waktu atau evaluasi yang setara serta ukur biaya validasi dan repair.

Pengembangan Genetic Algorithm dapat membandingkan versi tanpa elite dan dengan elite, mengukur diversitas populasi, dan menilai seberapa sering repair mengubah gen warisan. Pengembangan Simulated Annealing dapat mengevaluasi temperatur awal dan jadwal pendinginan pada anggaran yang sama. Solver eksak pada instance kecil berguna untuk menghitung gap optimum dan memeriksa apakah solusi relaksasi luas dapat diwujudkan secara geometris.

Untuk penggunaan operasional, model perlu diperluas berdasarkan kebutuhan perusahaan, misalnya distribusi berat, akses bongkar muat, dan jalur manuver. Perluasan tersebut harus disertai data dan constraint baru yang terdefinisi, sehingga dipisahkan dari pemenuhan spesifikasi tugas yang menggunakan grid dua dimensi dan satu kapal.

PEMBAGIAN TUGAS

NIM	Nama	Kontribusi
1824003	Keira Amelie Hanayesha	1. Perancangan arsitektur program 2. Implementasi representasi persoalan (core) 3. Pemodelan fungsi objektif utama dan bonus 4. Integrasi sistem 5. Penyusunan Laporan
18224071	Ellaine Juvina	1. Implementasi algoritma Genetic Algorithm 2. Analisis Perbandingan Algoritma 3. Visualisasi *Genetic Algorithm* 4. Penyusunan Laporan
18224119	Firanti Naulia Fasya	1. Implementasi algoritma Steepest Hill Climbing 2. Implementasi algoritma Stochastic Hill Climbing 3. Implementasi algoritma Hill Climbing with Sideways Move 4. Implementasi algoritma Random Restart Hill-Climbing 5. Implementasi algoritma Simulated Annealing 6. Penyusunan Laporan

REFERENSI

[1] Tim Asisten Laboratorium Inteligensi Artifisial, “Spesifikasi Tugas Besar 1 IF3070 Dasar Inteligensi Artifisial 2026/2027”, Versi 1.0, 24 September 2026.

[2] M. L. Khodra, "Beyond Classical Search: Classical vs Local Search," YouTube, 2021. [Video]. Available: Beyond Classical Search: Classical vs Local Search [Accessed: Sep. 16, 2026].

[3] M. L. Khodra, "Beyond Classical Search: State (Value, Neighbor)," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=_6rla37QorM. [Accessed: Sep. 16, 2026].

[4] M. L. Khodra, "Beyond Classical Search: Hill-climbing Search," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=L0b0CN3Fds8. [Accessed: Sep. 16, 2026].

[5] M. L. Khodra, "Beyond Classical Search: Simulated Annealing," YouTube, 2021. [Video]. Available: Beyond Classical Search: Simulated Annealing [Accessed: Sep. 16, 2026].

[6] M. L. Khodra, "Beyond Classical Search: Genetic Algorithm," YouTube, 2021. [Video]. Available: Beyond Classical Search: Genetic Algorithm [Accessed: Sep. 16, 2026].

LAMPIRAN FORM PENGGUNAAN GENERATIVE AI

FORMAT DEKLARASI PENGGUNAAN AI

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM : Keira Amelie Hanayesha / 18224003
: Ellaine Juvina / 18224071
: Firanti Naulia Fasya / 18224119
Sekolah : STEI - K / Sistem Teknologi Informasi
Kode Mata Kuliah : IF3070
Nama Mata Kuliah : Dasar Intelegensi Artifisial
Judul Tugas : Tugas Besar 1 – *Ship Loading Optimizer*

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	-
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	Sedang. Membantu menyusun draf laporan, merapikan paragraf, dan menyusun deskripsi implementasi algoritma.
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	Sedang. Membantu memahami konsep Local Search, Steepest-Ascent Hill Climbing, Simulated Annealing, fungsi, parameter, constraint, dan analisis implementasi.
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	-
Lainnya, Jelaskan:		
Diskusi dan penyempurnaan struktur kode pada algoritma Simulated		AI digunakan untuk memberikan

Annealing dan Genetic Algorithm.	masukan terhadap struktur fungsi, parameter awal, dan implementasi kode yang telah disusun. Proses dilakukan secara iteratif melalui evaluasi, pemberian saran perbaikan, dan penyesuaian kode berdasarkan hasil diskusi.
Penyusunan test case untuk menguji implementasi algoritma yang telah dibuat.	AI digunakan untuk membantu menyusun kode test case yang menguji implementasi algoritma berdasarkan berbagai variasi parameter dan batasan yang telah ditentukan, guna memeriksa kesesuaian dan kebenaran hasil implementasi.

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 22 Oktober 2026







Keira Amelie Hanayesha	Ellaine Juvina	Firanti Naulia Fasya
18224071	18224071	18224119